

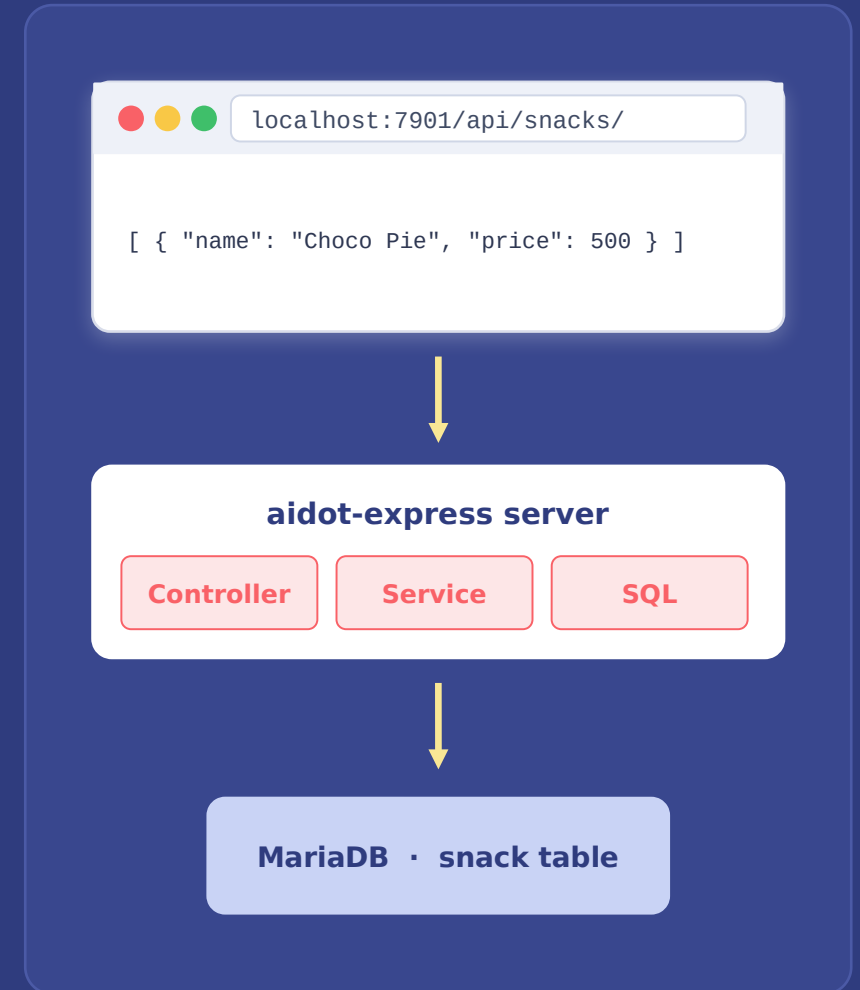
Follow along

Build a web server with aidot-express

Build a "class snack list" API from the admin console — by clicking

No code required. Press a few buttons and a real API appears.

aidot-express · Author: Mike



Part 1 : Build it as you read

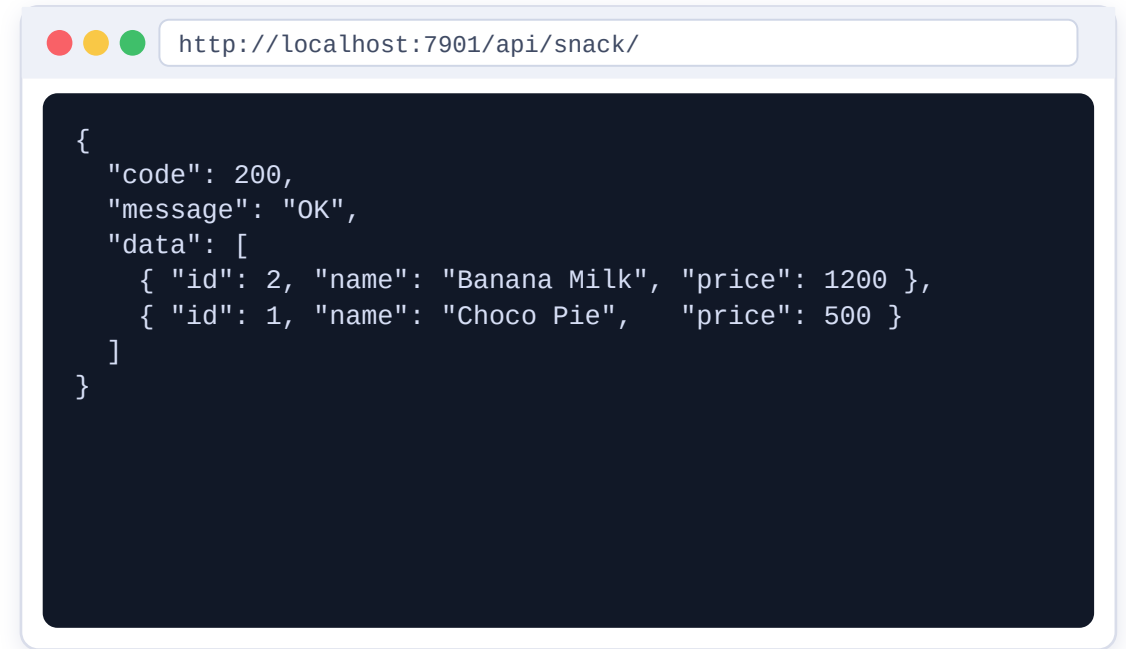
What you'll build

Five endpoints that store snack names and prices, and read them back

Five endpoints to build

GET	/api/snack/	List all snacks
GET	/api/snack/1	Read snack 1
POST	/api/snack/	Add a snack
PUT	/api/snack/1	Update snack 1
DELETE	/api/snack/1	Delete snack 1

Open the first one in a browser and you get this

A browser window with a white address bar showing the URL 'http://localhost:7901/api/snack/'. The main content area has a dark background and displays a JSON object. The JSON object has a 'code' property with value 200, a 'message' property with value 'OK', and a 'data' property which is an array of two objects. The first object in the array has 'id': 2, 'name': 'Banana Milk', and 'price': 1200. The second object has 'id': 1, 'name': 'Choco Pie', and 'price': 500.

```
{
  "code": 200,
  "message": "OK",
  "data": [
    { "id": 2, "name": "Banana Milk", "price": 1200 },
    { "id": 1, "name": "Choco Pie", "price": 500 }
  ]
}
```

That block of text is called JSON.
It is the shape programs use to talk to each other.

Steps 1 Create table → 2 SQL → 3 Service → 4 Controller → 5 Test

Install 1 — you were given the project folder

When you received the whole aidot-express folder

1 Check that Node.js is installed

Command prompt

```
C:\work\aidot-express> node -v  
v22.14.0      ← v20.19 or newer is fine
```

If nothing prints, install the LTS build from nodejs.org.

2 Install MariaDB and create the .env file

Create it with the command below, then fill in your database details by editing .env

Command prompt

```
C:\work\aidot-express> cp .env.example .env
```

3 Start it with npm start

Type a command and it runs.

Command prompt

```
C:\work\aidot-express> npm start
```

2-2 Open .env in Notepad and edit it

.env file

```
DB_TYPE=mariadb  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_USER=root  
DB_PASSWORD=admin  
DB_DATABASE=aidot_express
```

Command prompt

```
C:\work\aidot-express> npm start  
[start] Node.js v22.14.0 OK  
[start] root dependencies up to date  
[start] admin console (admin-client/dist) up to date  
[DB] mariadb connection OK  
[migration] done — 0 applied  
server started at http://localhost:7901
```

4

http://localhost:7901/

Admin console sign-in
(sign in as admin / admin1234)

Install 2 — you were given the installer (.exe)

No terminal, no commands. Double-click, then look for the tray icon.

1

Double-click the setup file

Aidot Express-Setup-1.32.0-x64.exe → "More info → Run anyway" → Next → Install

2

Enter ports and database

The defaults 7901 / 7902 are fine. Database type is mariadb.

3

It starts by itself

A splash appears briefly and the server starts in the background. No window is normal.

4

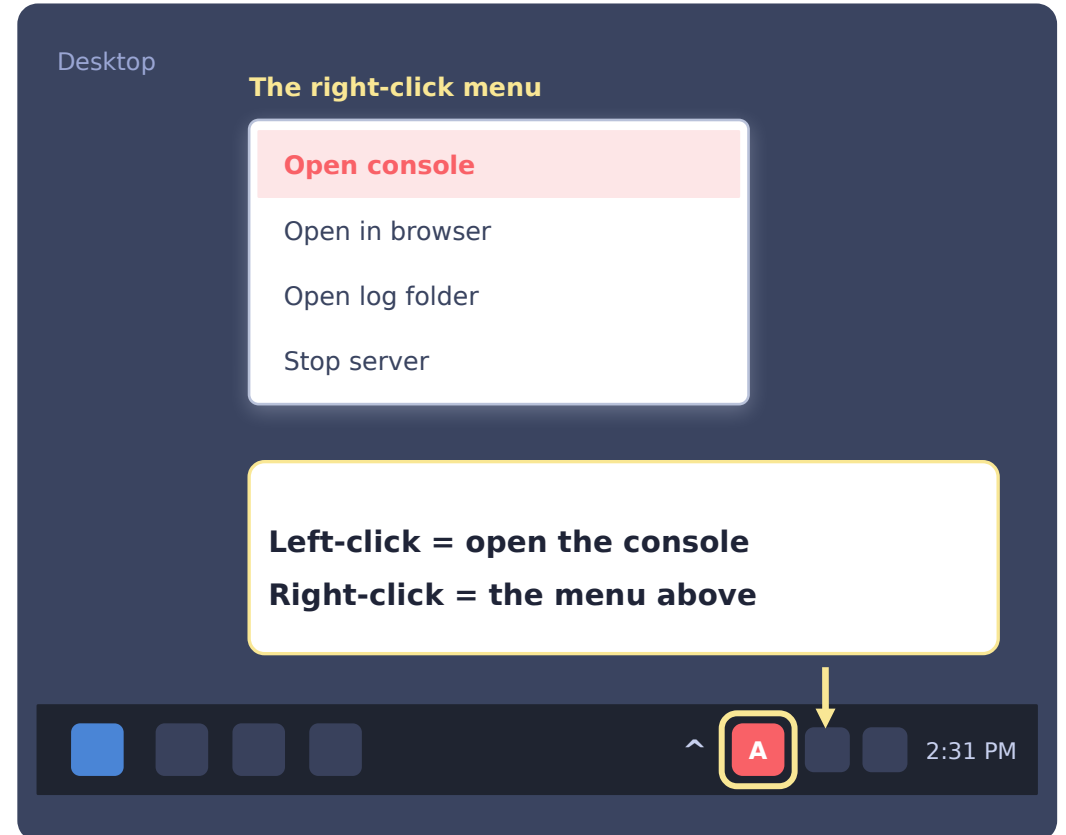
Find the tray icon

If you cannot see it, click the ^ next to the clock to show hidden icons.

Closing the window hides it in the tray rather than stopping it.

To really stop it, use the tray menu → Stop server.

Bottom right of the taskbar



Install

In short: run Setup → enter ports and database → install → click the tray icon → console opens (admin / admin1234)

Before you start — three things

If you finished install 1 or 2, steps 1 and 2 are already done — only step 3 is left.

1

Leave the server on

Run npm start in a command prompt and leave it running.

```
C:\work\aidot-express> npm start
```

2

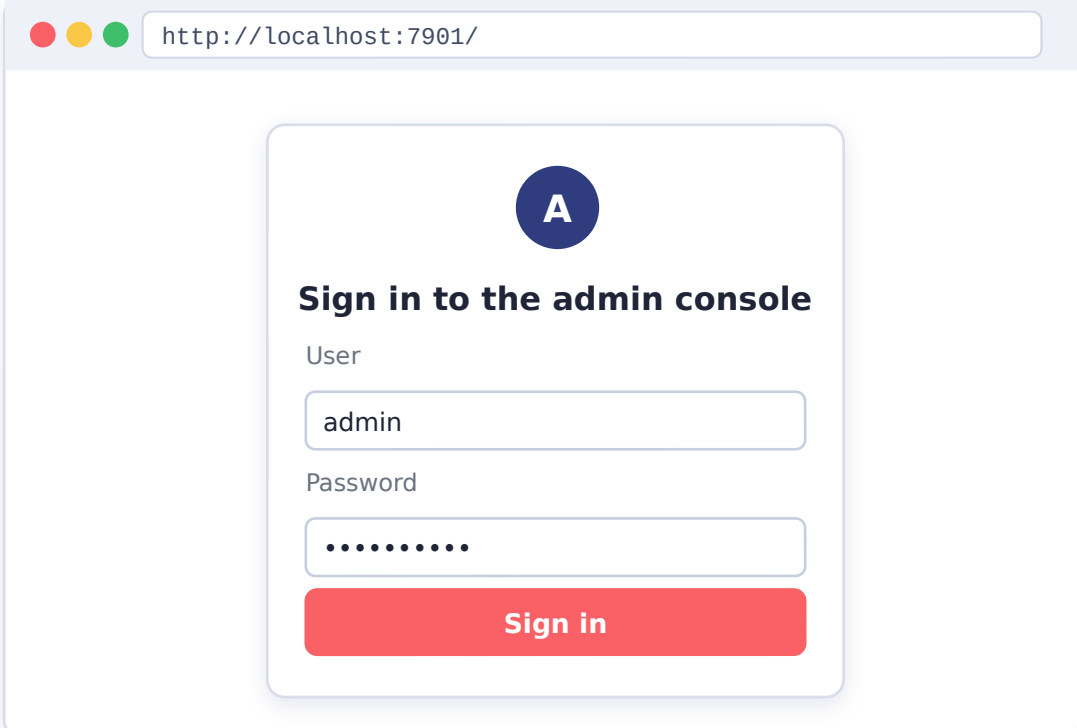
Sign in to the console

Go to localhost:7901 → user admin / password admin1234

3

Create one table for snacks

You need a table before you can store anything. The next page shows how.

A screenshot of a web browser window showing the admin console sign-in page. The address bar displays 'http://localhost:7901/'. The page features a central white box with a blue circle containing the letter 'A' at the top. Below this, the text 'Sign in to the admin console' is displayed. There are two input fields: 'User' with the value 'admin' and 'Password' with masked characters '.....'. A red 'Sign in' button is positioned at the bottom of the form.

A yellow banner asks you to change the password on first sign-in →

Change it under [Settings](#) → [My account](#) → [Change password](#).

Confirm

Ready when: the server is running · you are signed in · you can create a table.

A look around the console

Sign in with the default admin account

**aidot-express**
Spring-style Node.js Framework

구조는 Spring처럼, 속도는 번개처럼

- ✓ Spring 식 계층 구조 (Controller / Service / SQL)
- ✓ 서버 재시작 없는 핫 리로드
- ✓ 콘솔 내장 — SQL · 서비스 · API 테스트
- ✓ DB + 미들웨어(전문) 컨트롤러를 한 곳에서



```
graph TD; HTTP[HTTP 요청] --> Controller["@Controller<br/>라우트 매핑"]; Controller --> Service["@Service<br/>비즈니스 로직"]; Service --> Sql["@Sql<br/>쿼리 주입"]; Sql --> DB[(DB)];
```

Node.js · Express · Annotation based Routing



English **한국어**

로그인

개발도구 콘솔에 접속하려면 로그인하세요

사용자 ID

비밀번호

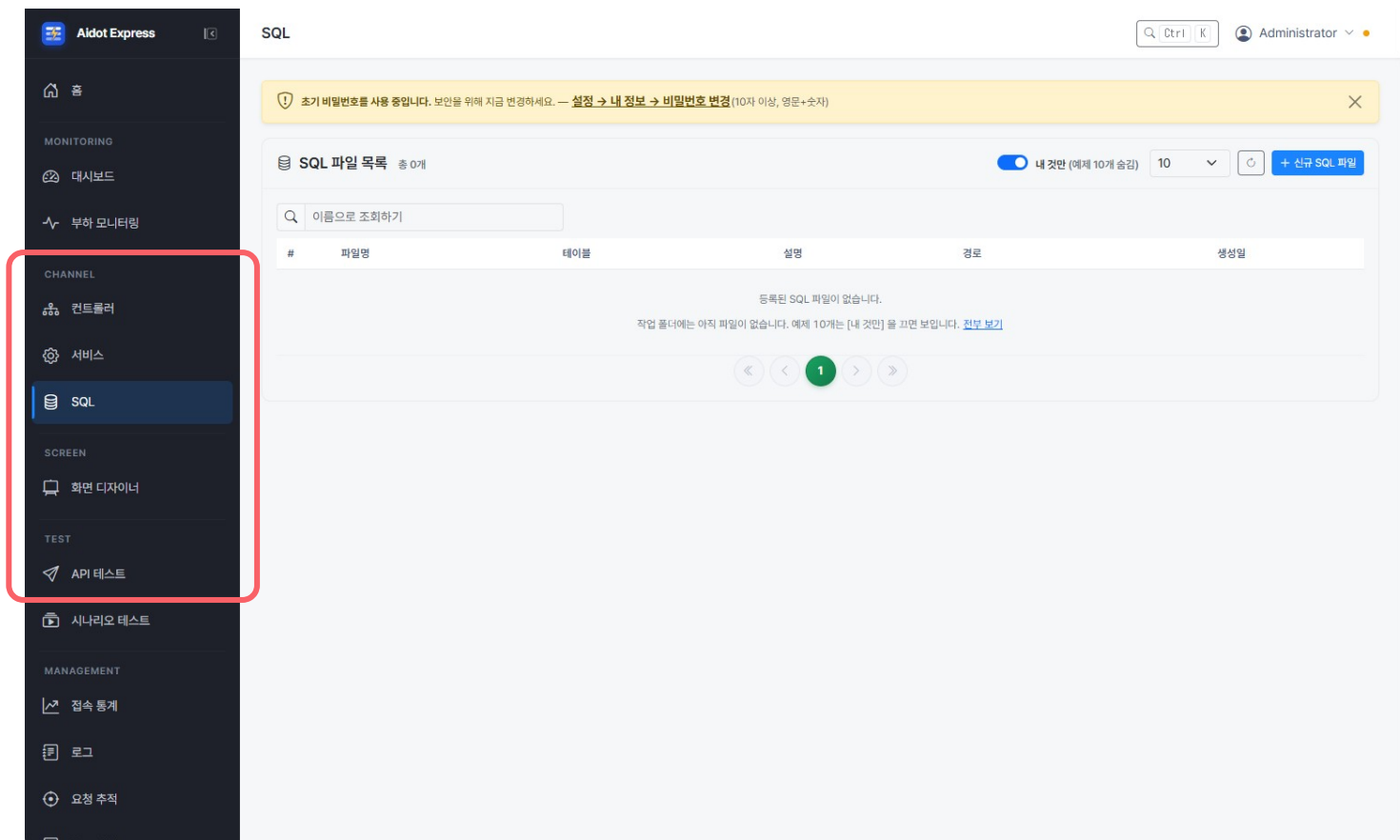
☐ 아이디 저장

로그인

DB 정상

A look around the console

The left menu is grouped (monitoring, channel, screen, test, management). You will use four of them.



The menus you will use

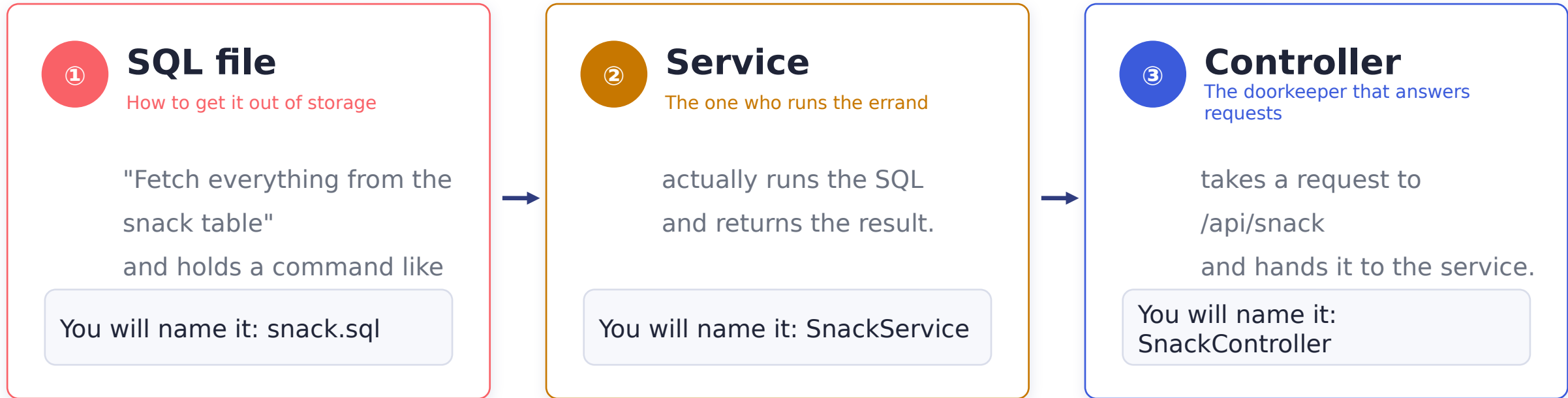
- 1 SQL**
Where you write how to get data out of storage
- 2 Service**
The errand runner that calls your SQL
- 3 Controller**
The doorkeeper that answers requests to a URL
- 4 API test**
Where you try the endpoints you built

Tip

Do not read the menu top to bottom — follow the build order: SQL → Service → Controller.

Before you build — three layers is all there is

Think of a school tuck shop



Tip

You build in that order too: 1 SQL → 2 Service → 3 Controller. Each one is needed to pick the next.

Step 0 — create a table to hold the snacks

You only do this once, in HeidiSQL (installed alongside MariaDB)

1

Open HeidiSQL, connect to the database, and click the Query tab

2

Select aidot-app in the tree on the left
Copy the text below and paste it in

```
CREATE TABLE snack (  
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  name        VARCHAR(50)  NOT NULL,  
  price       INT           NOT NULL DEFAULT 0,  
  memo        VARCHAR(200) NULL,  
  created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (id)  
)
```

3

Click run — one successful row below means it worked

The snack table appears under aidot-app on the left

This is the table you will create

snack (the snack table)

id	name	price	memo
1	Choco Pie	500	Most popular
2	Banana Milk	1200	After gym class
...			

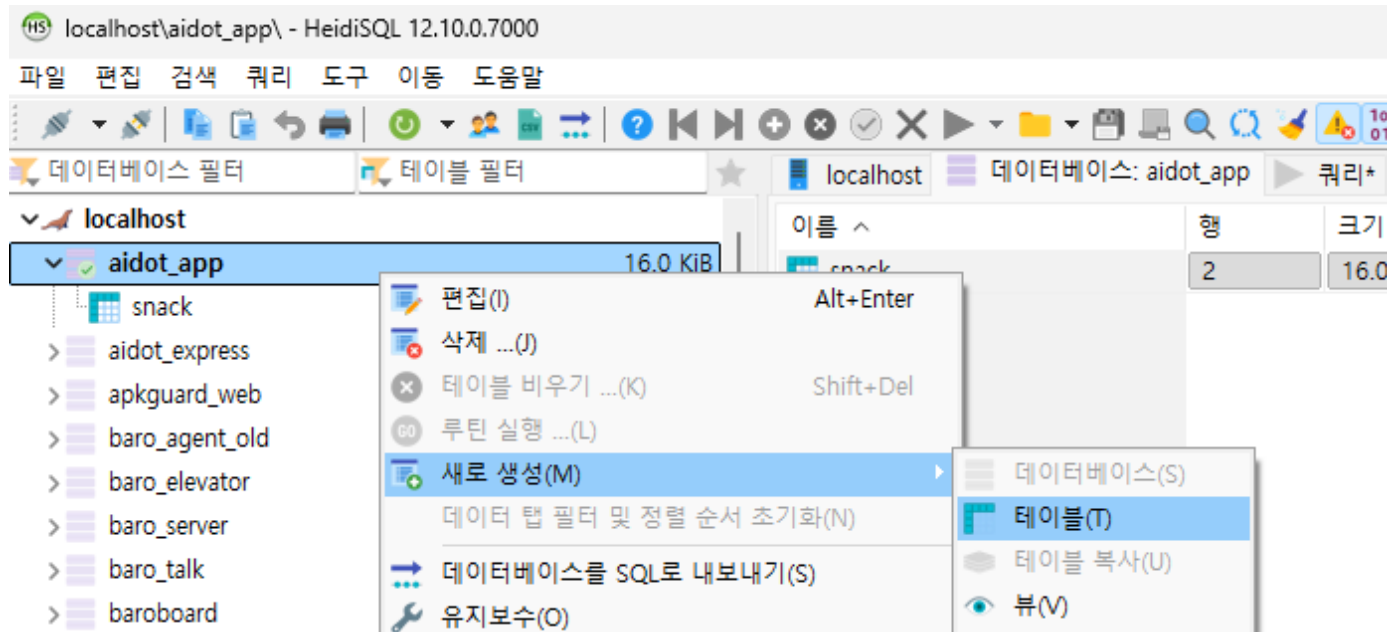
id counts up 1, 2, 3... on its own

Using SQLite?

Paste the same SQL into DB Browser for SQLite
and run it

Step 0, another way — create the table on screen

If writing SQL feels hard, you can build it with the mouse in HeidiSQL



**1 On the left,
right-click aidot_app
→ New ► Table**

Steps

Click order: right-click the database → New ► Table → name it snack → add 5 columns → Save

Step 0, another way — create the table on screen

If writing SQL feels hard, you can build it with the mouse in HeidiSQL

The screenshot shows the HeidiSQL interface with the 'snack' table being created in the 'aidot_app' database. The table structure is defined as follows:

#	이름	데이터 유형	길이/설정	부호 ...	NULL ...	0으...	기본값	코멘트
1	id	BIGINT	20	☒	☐	☐	AUTO_INCREME...	
2	name	VARCHAR	50	☐	☐	☐	기본값 없음	
3	price	INT	11	☐	☐	☐	'0'	
4	memo	VARCHAR	200	☐	☒	☐	NULL	
5	created_at	DATETIME		☐	☐	☐	current_timestam...	

The bottom panel shows the SQL code generated for the table creation:

```
22 /* "C:\Users\mikej\AppData\Roaming\HeidiSQL\Backups\query-tab-2026-04-15_20-37-52-667.sql" (287 B) 파일을 쿼리 탭 #1 으로 불러오는 중 ... */
23 /* "C:\Users\mikej\AppData\Roaming\HeidiSQL\Backups\query-tab-2026-08-29_06-18-48-213.sql" (308 B) 파일을 쿼리 탭 #2 으로 불러오는 중 ... */
24 /* 화면 DPI에 대한 크기 조정 제어: 100% */
25 SELECT * FROM `information_schema`.`COLUMNS` WHERE TABLE_SCHEMA='aidot_app' AND TABLE_NAME='snack' ORDER BY ORDINAL_POSITION;
26 SHOW INDEXES FROM `snack` FROM `aidot_app`;
27 SELECT * FROM information_schema.REFERENTIAL_CONSTRAINTS WHERE CONSTRAINT_SCHEMA='aidot_app' AND TABLE_NAME='snack' AND REFERENCED_TABLE_NAME IS NOT NULL;
28 SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA='aidot_app' AND TABLE_NAME='snack' AND REFERENCED_TABLE_NAME IS NOT NULL;
29 SELECT FULL_COLLATION_NAME AS `Collation`, CHARACTER_SET_NAME AS `Charset`, ID AS `Id`, IS_DEFAULT AS `Default`, 0 AS `Sortlen` FROM `information_schema`.COLLATION_CHARAC
30 SHOW ENGINES;
31 SHOW CREATE TABLE `aidot_app`.`snack`;
32 SELECT CONSTRAINT_NAME, CHECK_CLAUSE FROM `information_schema`.`CHECK_CONSTRAINTS` WHERE CONSTRAINT_SCHEMA='aidot_app' AND TABLE_NAME='snack';
33 SHOW CREATE TABLE `aidot_app`.`snack`;
```

2 Type snack as the name
Use + Add to make five columns.

Exactly the columns shown on the left.

3 Save, at the top right
(or Ctrl+S)
snack appears in the list on the left — done.

Step 0 — add some snacks

You only do this once, in HeidiSQL

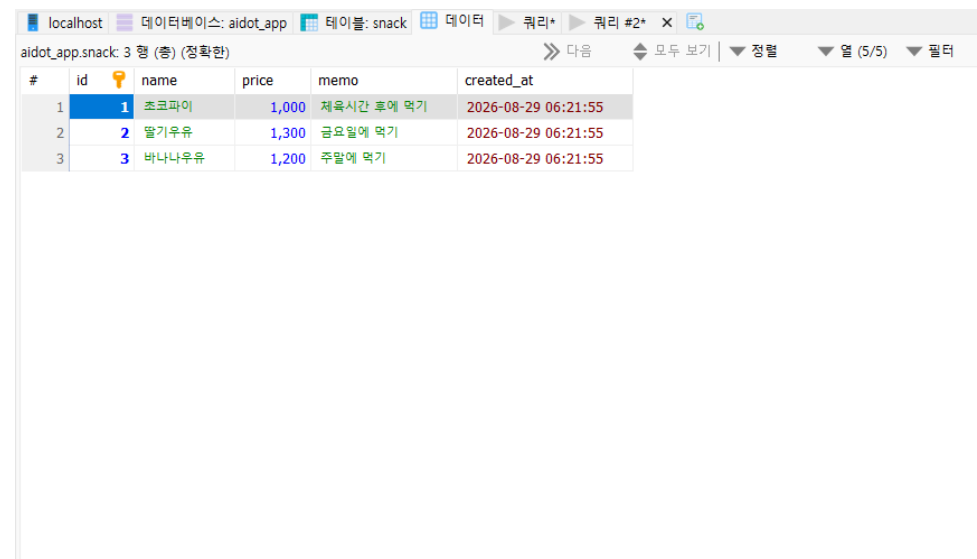
1 Open HeidiSQL and click the Query tab

2 Copy the text below and paste it in

```
INSERT INTO `snack` (`name`, `price`, `memo`) VALUES ('Choco Pie', 1000, 'After gym class');
INSERT INTO `snack` (`name`, `price`, `memo`) VALUES ('Strawberry Milk', 1300, 'Save for Friday');
INSERT INTO `snack` (`name`, `price`, `memo`) VALUES ('Banana Milk', 1200, 'Save for the weekend');
```

3 **Click run — one successful row below means it worked**
Then open the Data tab to see the rows

This is the data you end up with



The screenshot shows the HeidiSQL interface with the 'snack' table selected. The table has 3 rows of data. The columns are: #, id, name, price, memo, and created_at. The data is as follows:

#	id	name	price	memo	created_at
1	1	초코파이	1,000	체육시간 후에 먹기	2026-08-29 06:21:55
2	2	딸기우유	1,300	금요일에 먹기	2026-08-29 06:21:55
3	3	바나나우유	1,200	주말에 먹기	2026-08-29 06:21:55

Using SQLite?

Paste the same SQL into DB Browser for SQLite
and run it

Step 0, another way — add snacks on screen

Fill the cells in the Data tab like a spreadsheet and it saves as typed

HeidiSQL — aidot_app.snack

Table **Data** Query **+ Insert row** Delete row Save F5

id	name	price	memo	created_at
1	Choco Pie	500	Most popular	2026-08-24 09:03
2	Banana Milk	1200	After gym class	2026-08-24 09:12
(auto)	Strawberry Milk	1300	Save for Friday	(auto)

Leave id and created_at empty — the server fills them in when you save.

After saving you see this below

```
/* rows affected: 1 */ INSERT INTO `snack` (`name`, `price`, `memo`) VALUES ('Strawberry Milk', 1300, 'Save for Friday');
```

1 Click the Data tab at the top

2 Click + Insert row and an empty row appears at the bottom.

**3 Click a cell and type
Tab moves to the next cell**

**4 Save when you are done
(or Ctrl+S)**

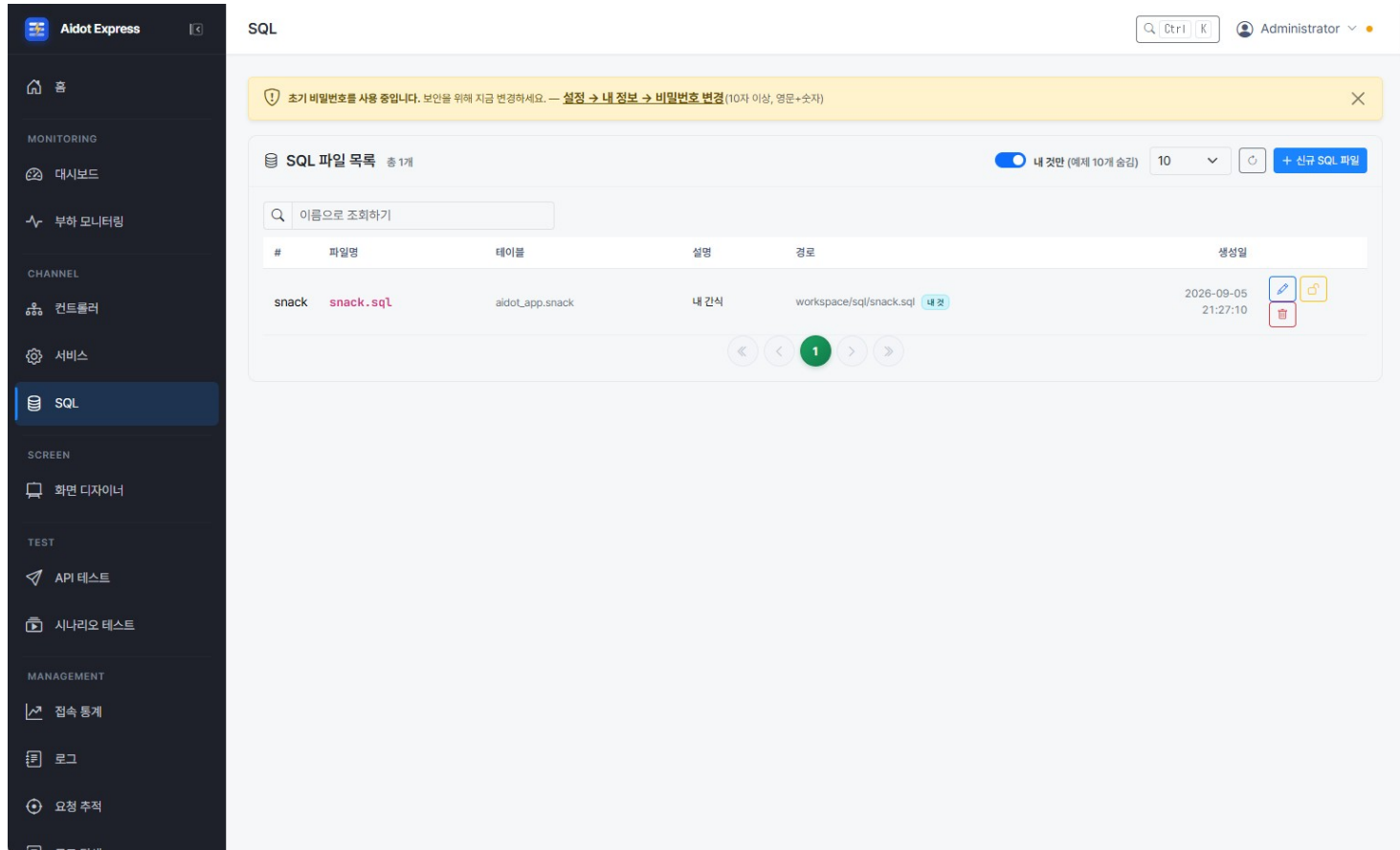
F5 reloads it.
id counts up 1, 2, 3...
Check that it stuck.

Steps

Click order: Data tab → + Insert row → fill the cells (Tab to move) → Save → F5 to check

Step 1 — create the SQL file (1)

Click SQL in the left menu for the list, then the blue + New SQL file at the top right



The list starts out empty.

The blue button at the top right is where you start.

With "mine only" on,

you see only what you made.

Turn it off to see the examples too.

Once created, this list

gains a row.

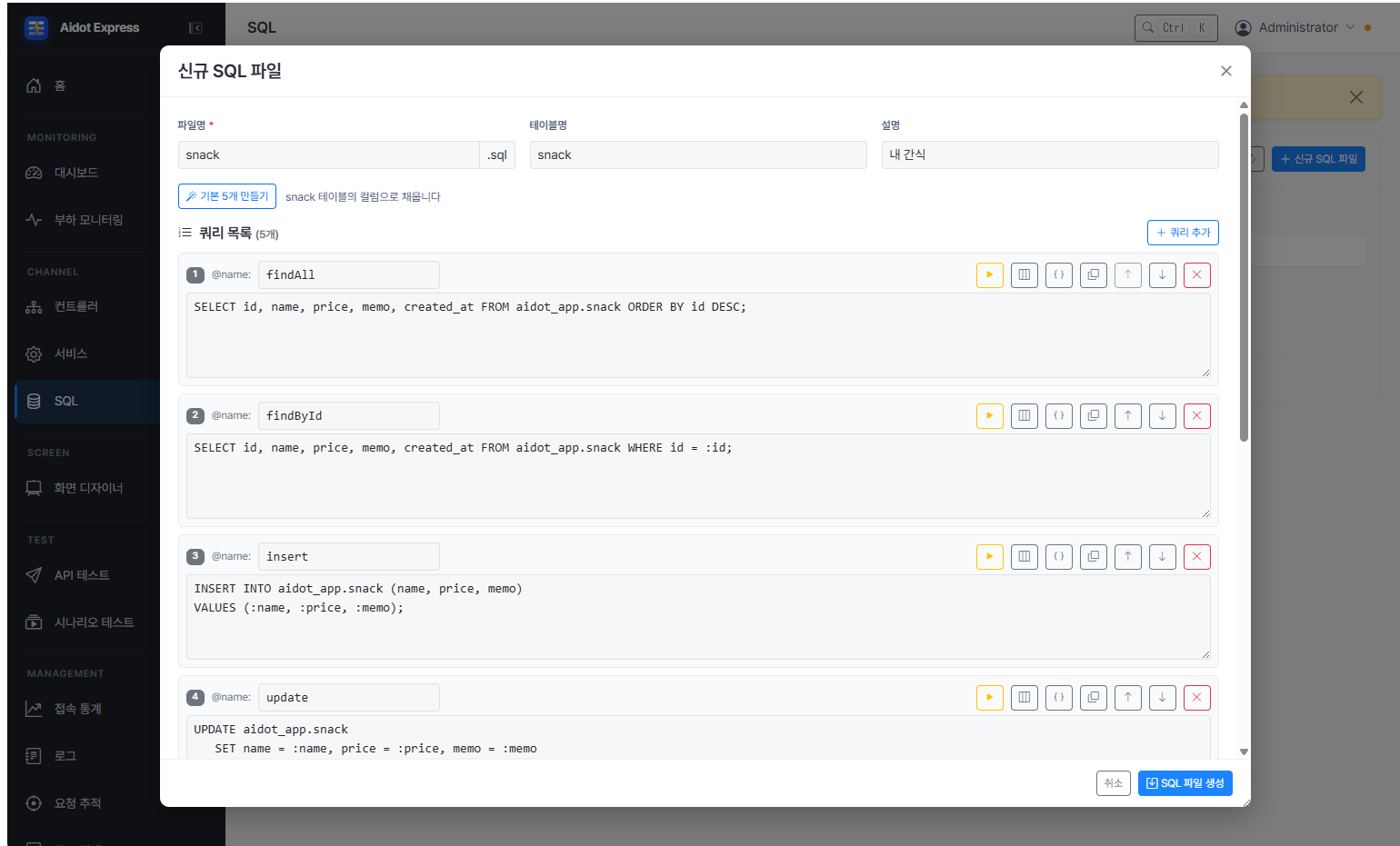
(the path reads workspace/sql/...)

Steps

Click order: SQL on the left → + New SQL file at the top right

Step 1 — create the SQL file (2)

Type the file and table names, click "Create 5 basics", and five queries appear at once



File name is snack
(.sql is added for you)

You need the table name for
the columns are filled in too.
If you set a schema,
it reads aidot_app.snack.

Click "Create 5 basics" and
five queries appear in the list below.
(findAll · findById · insert ·
update · deleteById)

Enter Type: file name snack · table name snack → Create 5 basics → Create SQL file

Step 1 — create the SQL file (3)

"Create 5 basics" writes all five below at once — you type no code

```
-- @name: findAll
SELECT id, name, price, memo, created_at
FROM aidot_app.snack
ORDER BY id DESC;

-- @name: findById
SELECT id, name, price, memo, created_at FROM aidot_app.snack WHERE id = :id;

-- @name: insert
INSERT INTO aidot_app.snack (name, price, memo)
VALUES (:name, :price, :memo);

-- @name: update
UPDATE aidot_app.snack
  SET name = :name, price = :price, memo = :memo
  WHERE id = :id;

-- @name: deleteById
DELETE FROM aidot_app.snack WHERE id = :id;
```

Two things to know

-- @name: findAll

is the label on each query.

When the service calls this name,
the SQL just below runs.

:id :name :price

is a blank that a value fills later.

A value sent from the browser lands here
and it is filled in safely.

The five labels are fixed names.

You pick them as-is in the next step.

Next

After creating them, click Run test then Save — otherwise you cannot pick them next.

Step 1 — create the SQL file (4)

Click the ► on the right of a query card and the result opens just below

The screenshot shows the Aidot Express SQL interface. A modal window titled 'SQL 파일 수정' (Edit SQL File) is open, displaying a query card for 'findAll'. The query is: `SELECT id, name, price, memo, created_at FROM aidot_app.snack ORDER BY id DESC;`. Below the query, the results are shown in a table with 3 rows and 5 columns: id, name, price, memo, and created_at. The table contains data for three items: id 3 (바나나우유), id 2 (딸기우유), and id 1 (초코파이). The interface also shows a sidebar with navigation options like 'MONITORING', '대시보드', '부하 모니터링', 'CHANNEL', '컨트롤러', '서비스', 'SQL', 'SCREEN', '화면 디자이너', 'TEST', 'API 테스트', '시나리오 테스트', 'MANAGEMENT', '접속 통계', '로그', '요청 추적', and '로그 상세'.

id	name	price	memo	created_at
3	바나나우유	1200	주말에 먹기	2026-08-28T21:21:55.000Z
2	딸기우유	1300	금요일에 먹기	2026-08-28T21:21:55.000Z
1	초코파이	1000	제육시간 후에 먹기	2026-08-28T21:21:55.000Z

1 Find the query card you want to check

2 Click the yellow ► on the right (the icons beside it: columns, variables, copy, order, delete)

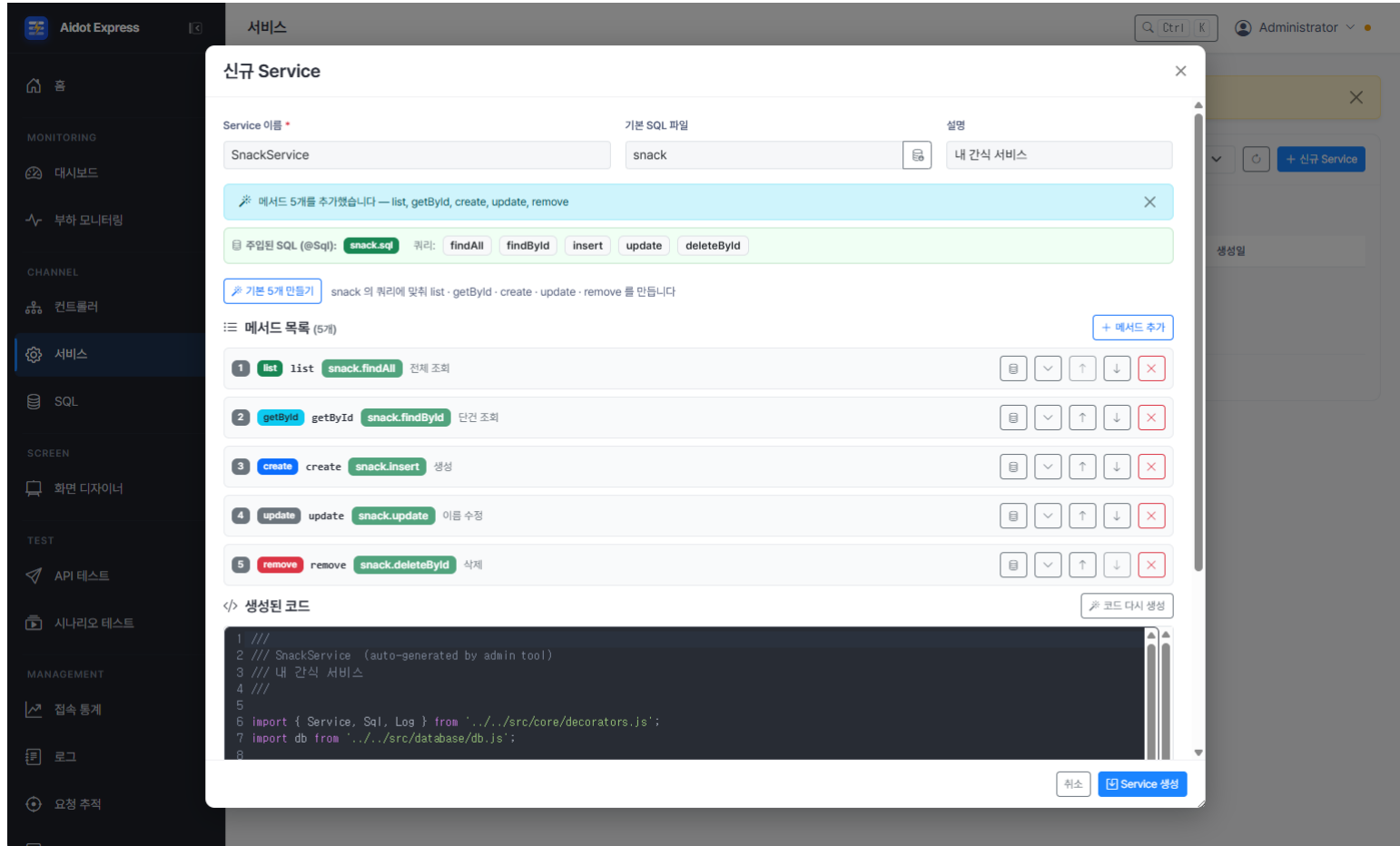
The result opens under that card.
Rows in a table means it worked.
Red text means a typo — check it.

Check the rows it returned.
It also shows the SQL it ran.

Steps Click order: ► on the query card → Run below → check the rows

Step 2 — create the service

Type a name, pick the default SQL file, click "Create 5 basics", and the methods appear



Name it SnackService

(capital first letter, no spaces)

Method: something you can run on its own

Pick snack as the default SQL file and that file's queries show as green badges.

Click "Create 5 basics" and the methods appear.
list · getById · create · update and remove — five in all.

Enter Type: name SnackService · default SQL file snack → Create 5 basics → Create service

Step 3 — create the controller (1)

Type a name and base path, then pick the default service

신규 컨트롤러

컨트롤러 이름 * SnackController 유형 DB 기본 요청 경로 * /api/snack 기본 Service SnackService

설명

내 간식 API

☐ 전체 인증 필요 OFF — 개별 라우트에서 설정 가능

☐ 실시간 알림 (SSE) 커먼 구독 주소가 자동으로 추가되고, 데이터를 넣거나 고치거나 지울 때 접속한 화면에 즉시 알려 줍니다.

기본 5개 만들기 list · getById · create · updateName · remove

라우트 목록 (0개) + 라우트 추가

라우트가 없습니다. 위의 '5개 기본 라우트 자동 생성'으로 한 번에 만들거나, 오른쪽 [라우트 추가]로 하나씩 만드세요.

생성된 코드 코드 다시 생성

1

취소 컨트롤러 생성

Name it SnackController
Route: something the browser
something you can call
on its own

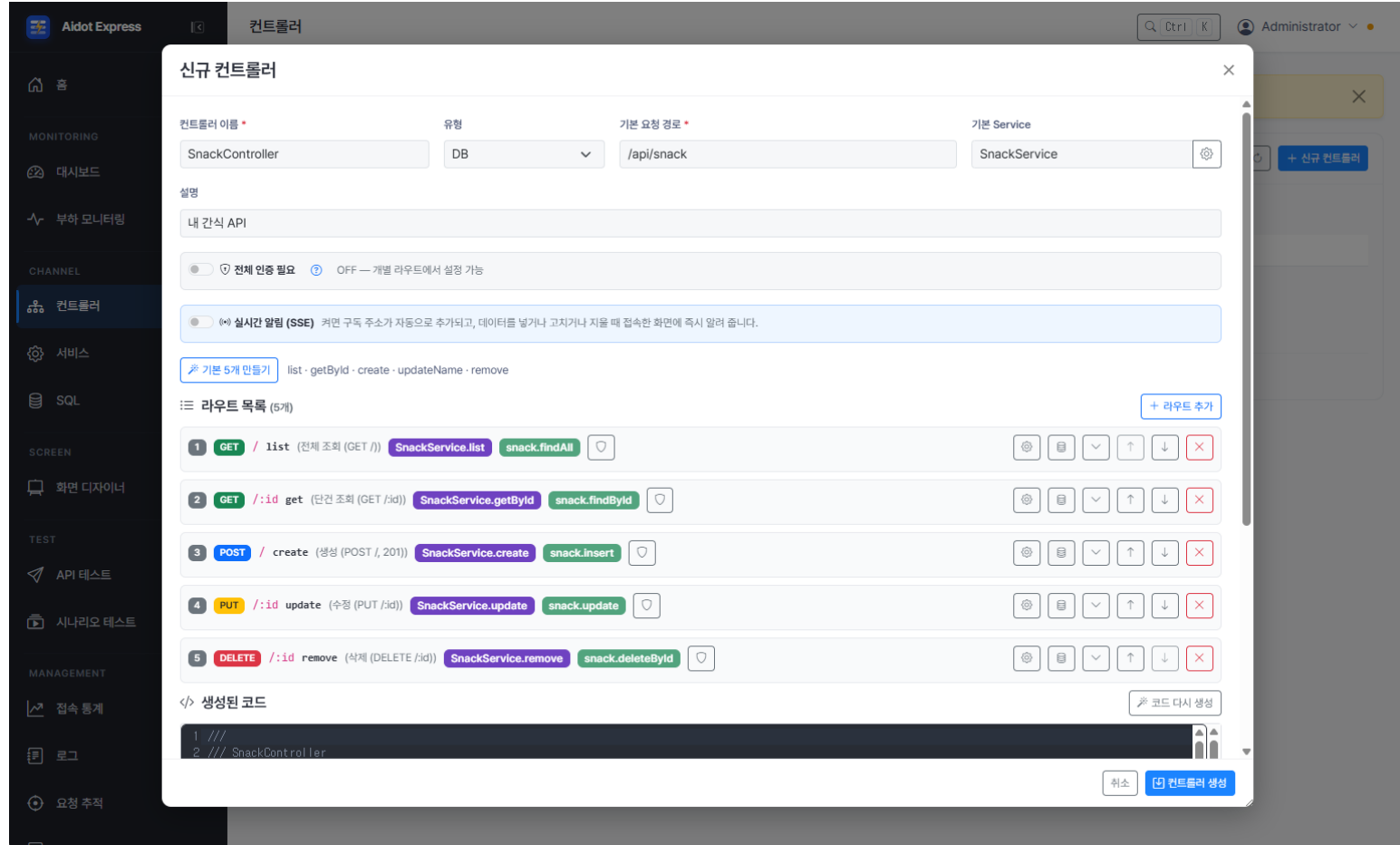
The base path is the address.
/api/snack →
localhost:7901/api/snack/

Enter SnackService as the
default service.

Enter **Type: name SnackController · path /api/snack · default service SnackService**

Step 3 — create the controller (2)

One click on "Create 5 basics" writes five routes and their code



Click "Create 5 basics" and
Five routes appear,
and the generated code below
is written at the same time.

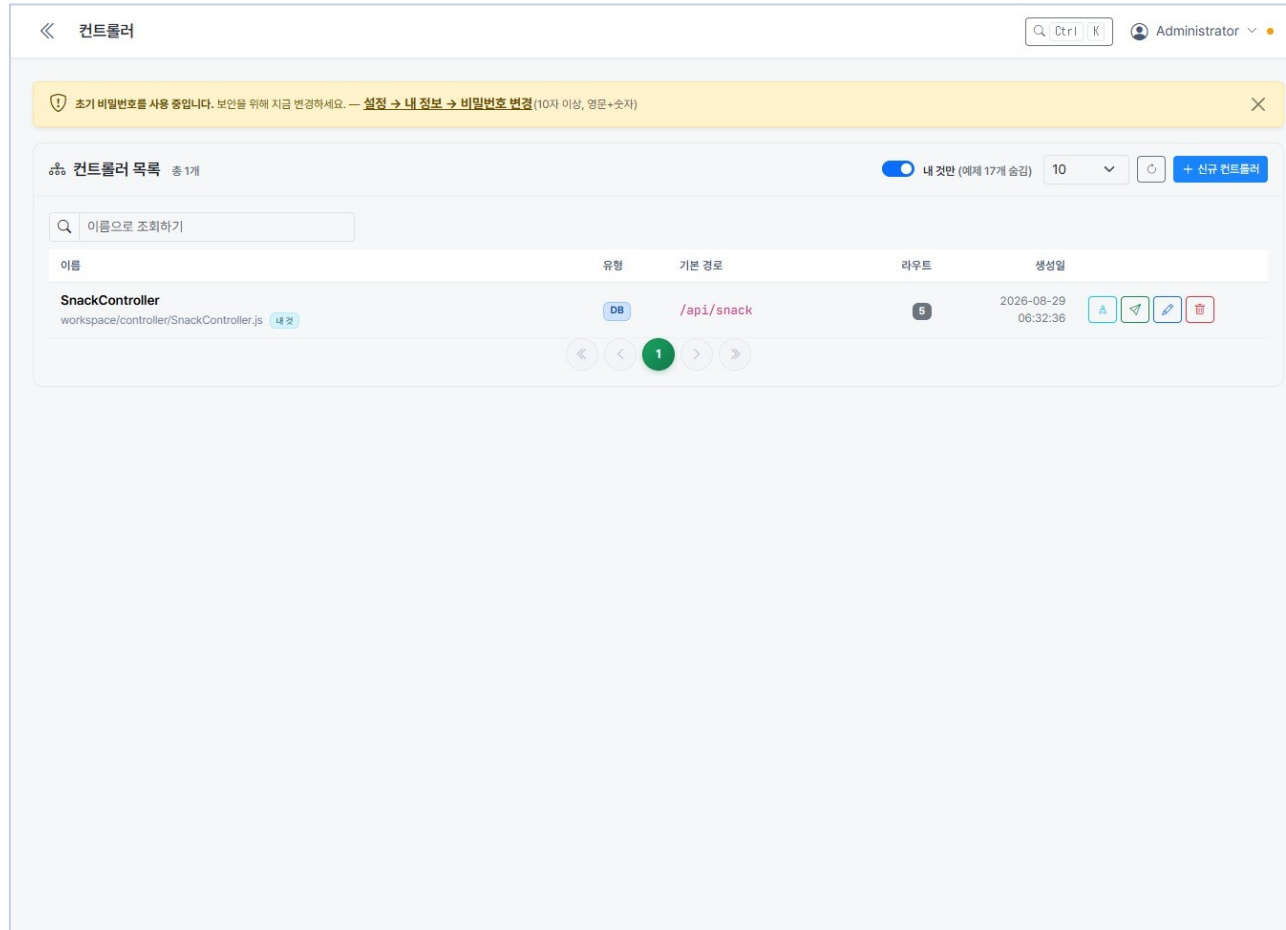
**The green and purple badges
show**
"which service and SQL this route
calls".

Lines with `/:id` take
a number in the address.
`/api/snack/2` → id is 2

Steps Click order: Create 5 basics → check the five routes and code → Create controller

Step 3 — check that it worked (3)

A new row with a route count means the API is already live

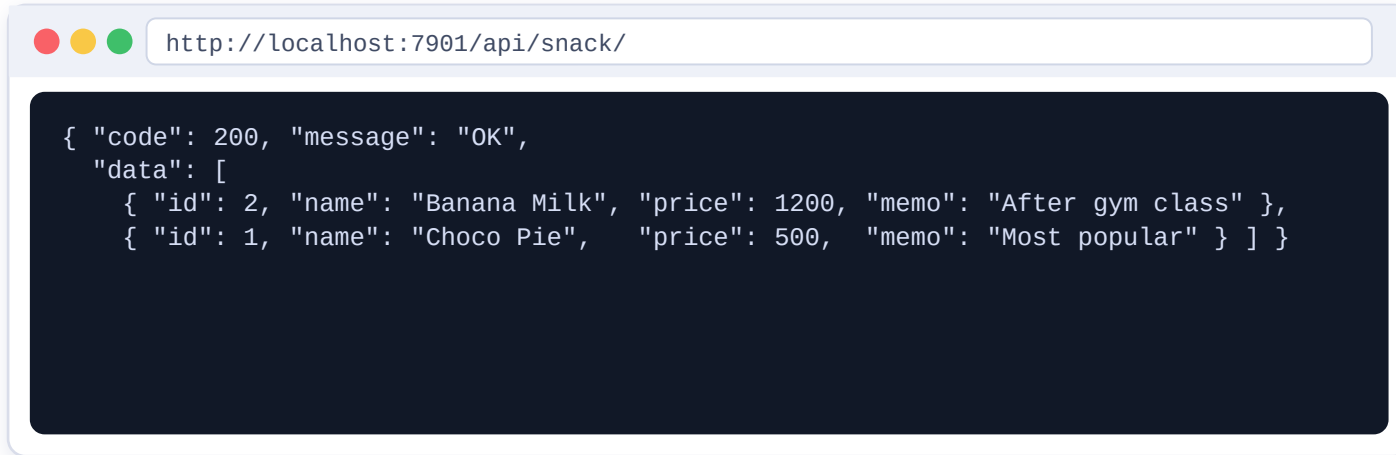


Without restarting the server,
it works right away.

with the lock icon.
You can also disable this controller
for a while (the file is not deleted).

Step 4 — check it in a browser

Type it into the address bar. GET requests can be checked from a browser.



```
{ "code": 200, "message": "OK",  
  "data": [  
    { "id": 2, "name": "Banana Milk", "price": 1200, "memo": "After gym class" },  
    { "id": 1, "name": "Choco Pie", "price": 500, "memo": "Most popular" } ] }
```



```
{ "code": 200, "message": "OK",  
  "data": { "id": 1, "name": "Choco Pie", "price": 500, "memo": "Most popular" } }
```

Two things to check



Is code 200?

200 means it worked.



Are the snacks inside data?

The snacks you put in the table appear — that is it.

Everything crammed together?

Install the JSON Viewer extension and
it lines up neatly.

**POST, PUT and DELETE cannot be tested from
the address bar → see the next page.**

Step 5 — add a snack from API test

You can send a POST from inside the console — no extra program to install

aidot-express admin console

admin

Home

Dashboard

Controller

Service

SQL

API test

Logs

Users

POST ▾

/api/snack/

Send

Body (what you send)

```
{  
  "name": "Banana Milk", "price": 1200, "memo": "After gym class"  
}
```

Response

201

```
{ "code": 201, "message": "Created",  
  "data": { "insertId": 2, "rowsAffected": 1 } }
```

1 Set the method to POST

2 Address is /api/snack/

3 Type the body

4 Send

insertId: 2 means

"saved as number 2"

rowsAffected: 1

means "one row was inserted".

Now, in the browser,

With /api/snack/ in the address

Refresh and the snack you just added is there.

Steps

Click order: API test → POST → /api/snack/ → type the body → Send → check for 201

Step 6 — test with Postman

Postman is one of the most common ways to test web requests. (postman.com → install → New → HTTP Request)

Postman Add a snack

POST `http://localhost:7901/api/snack/` **Send**

Params Headers **Body** raw JSON

```
{
  "name": "Strawberry Milk", "price": 1300, "memo": "Save for Friday"
}
```

Response **201 Created** · 34 ms

```
{ "code": 201, "message": "Created",
  "data": { "insertId": 3, "rowsAffected": 1 } }
```

Try all five like this

Method	Address	Body needed?
GET	/api/snack/	No
GET	/api/snack/1	No
POST	/api/snack/	Yes (name and price)
PUT	/api/snack/1	Yes (what to change)
DELETE	/api/snack/1	No

**When you send a body, pick
[Body] → [raw] → [JSON]**

If you prefer the command line, there is curl.
One line does it.

```
curl localhost:7901/api/snack/
```

Part 2 : Build it by hand

You will do the same thing by hand, with three files.

Looking inside the code shows you what the console was doing for you.

1

`workspace/sql/supply.sql`

SQL file — how to get data out of storage

2

`workspace/service/SupplyService.js`

Service — the errand runner that executes the SQL

3

`workspace/controller/SupplyController.js`

Controller — the doorkeeper that takes requests

Names: console = snack · by hand = supply · shipped example = guestbook

By hand (1) — create the table

Create the table and add rows with HeidiSQL.

```
CREATE TABLE supply (  
  id    BIGINT AUTO_INCREMENT,  
  name  VARCHAR(50) NOT NULL,  
  count INT DEFAULT 1,  
  memo  VARCHAR(200),  
  PRIMARY KEY (id));
```

```
INSERT INTO aidot_app.supply (name, count, memo) VALUES  
  ('Pencil',      12, '2B'),  
  ('Notebook',    5, 'Ruled'),  
  ('Eraser',      3, NULL),  
  ('Pencils',     24, 'Two 12-colour tins'),  
  ('Scissors',    2, 'Safety'),  
  ('Glue',        4, 'Stick'),  
  ('Ruler',       1, '30cm'),  
  ('Paints',      1, '12 colours'),  
  ('Sketchbook',  2, 'B4'),  
  ('Pencil case', 1, NULL);
```

By hand (1) — the SQL file

Create workspace/sql/supply.sql

```
-- our class supply list

-- @name: findAll
SELECT id, name, count, memo FROM aidot_app.supply ORDER BY id DESC;

-- @name: findById
SELECT id, name, count, memo FROM aidot_app.supply WHERE id = :id;

-- @name: insert
INSERT INTO aidot_app.supply (name, count, memo)
VALUES (:name, :count, :memo);

-- @name: update
UPDATE aidot_app.supply SET name = :name, count = :count, memo = :memo
WHERE id = :id;

-- @name: deleteById
DELETE FROM aidot_app.supply WHERE id = :id;
```

Open VS Code and create the file

1

Write the label first

-- write the query name after -- @name:

2

Semicolon at the end

End each statement with a semicolon.

3

A value slot is written :name

Prefix them with a colon, like :id and :name.

You need a supply table too

Run this in HeidiSQL first (same as step 0)

```
CREATE TABLE supply (
  id    BIGINT AUTO_INCREMENT,
  name  VARCHAR(50) NOT NULL,
  count INT DEFAULT 1,
  memo  VARCHAR(200),
  PRIMARY KEY (id));
```

Name

snack came from the console, supply comes from files — different topics, no name clashes.

By hand (2) — the service file

Create workspace/service/SupplyService.js

```
import { Service, Sql, Log } from '../core/decorators.js';
import db from '../database/db.js';

@Service('SupplyService')          // callable under this name
export default class SupplyService {

  @Sql('supply') supplySql;         // uses the supply.sql file
  @Log log;                         // gives you a logger

  async list() {                   // read all
    const sql = this.supplySql.get('findAll'); // fetch by label
    const res = await db.execute(sql, {});      // run the SQL
    return res.rows;                          // hand it to the controller
  }

  async getById(id) {              // read one
    const sql = this.supplySql.get('findById');
    const res = await db.execute(sql, { id }); // fill the :id slot
    return res.rows[0] ?? null;
  }
}
```

Three pieces of code matter

@Service

a label that tells the server

"I am an errand runner"

@Sql('supply')

pulls in the whole

supply.sql file

db.execute(sql, {...})

actually runs the SQL

and takes the result

create, update and remove follow the same shape.

By hand (3) — the controller file

Create workspace/controller/SupplyController.js and the address opens

```
import {
  Controller, Log, Autowired,
  GetMapping, PostMapping, DeleteMapping,
} from '../core/decorators.js';

@Controller('/api/supply') // ← served at this address
export default class SupplyController {

  @Autowired('SupplyService') supplyService; // call the errand runner
  @Log log;

  @GetMapping('/') // GET /api/supply/
  async list(params) {
    return this.supplyService.list();
  }

  @GetMapping('/:id') // GET /api/supply/1
  async get(params) {
    return this.supplyService.getById(params.id);
  }
}
```

This time the address is /api/supply, not /api/snack.

Saving alone does not apply it

- A newly added controller can be picked up while the server is running.
- You can restart the server from the dashboard.
- To restart everything,
Or press Ctrl+C and start it again with npm start.

Check it

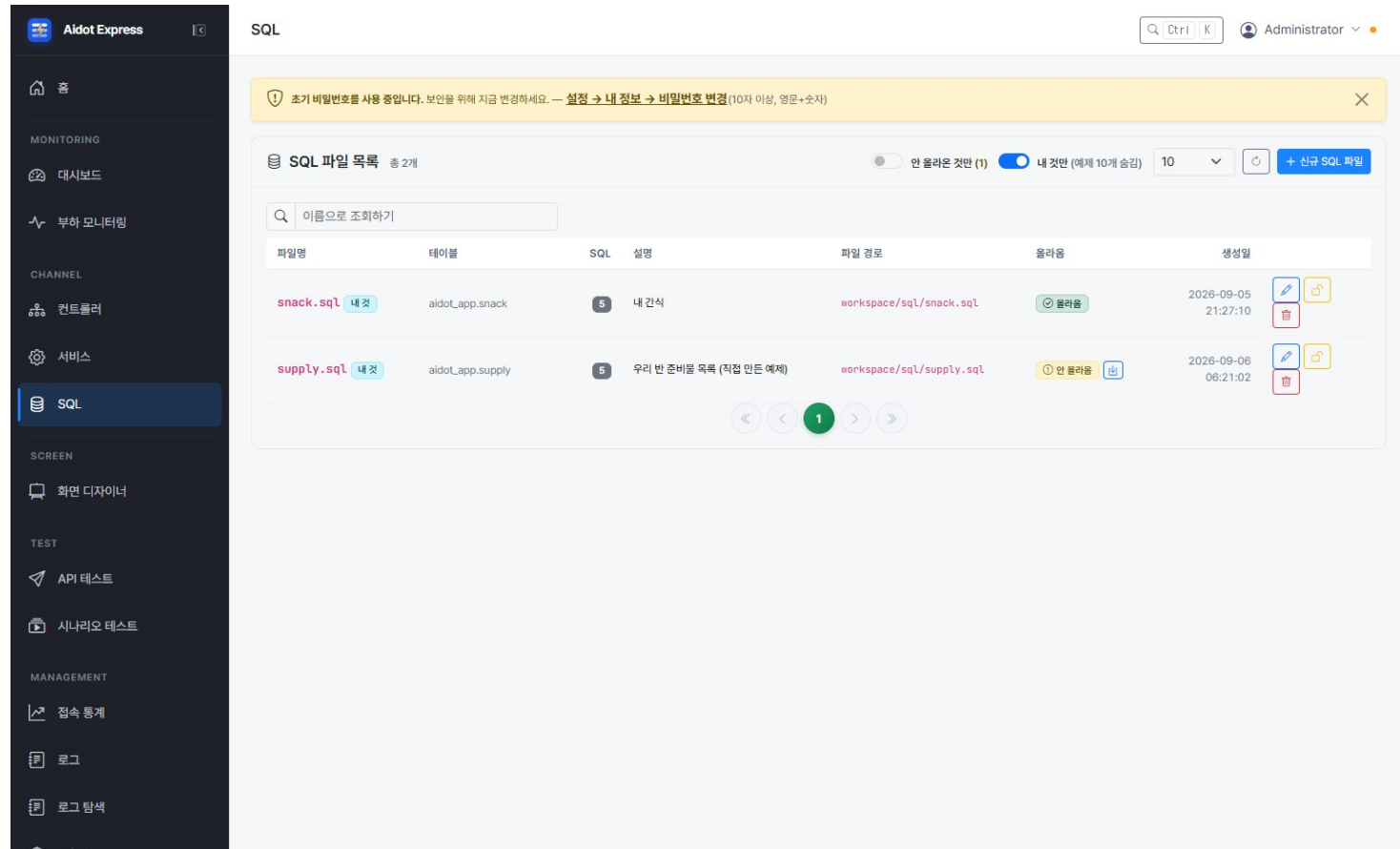
```
localhost:7901
  /api/supply/
```

In the console Controllers list,
SupplyController appears.

Built in the console, built by hand —
To the server they are the same.

By hand — load them from the console

In the SQL, Services and Controllers lists, load whatever shows "Not loaded"



The Loaded column

- Loaded items show "Loaded", ones that are not show "Not loaded"

Load

- Click "Load this one" and only that item loads

Step 7 — try it from a web page (HTML)

This guestbook page already ships with the server — just type the address

http://localhost:7901/public/demo/guestbook-lab.html

Guestbook API playground

GET 1 Read all entries

Load all

GET 2 Read one

Load

POST 3 Leave an entry

Leave an entry

DELETE 5 Delete an entry

Delete

What the server sent back

```
{
  "code": 200,
  "data": [
    {
      "id": 2,
      "writer": "Alex Kim",
      "message": "First visit!"
    }
  ]
}
```

Type it into the address bar

localhost:7901
/public/demo
/guestbook-lab.html

The guestbook example ships with the server. You can point it at your own `/api/snack` instead.

Each time you click a button, the response on the right changes, The time it took (ms) is shown too.

Steps 1 Type the page address → 2 Click a button → 3 Read the response on the right

Step 7 (1) — build your own page in VS Code

Paste it into one empty file — it loads the snack list

- 1 Open VS Code → File → Open Folder
Any folder will do (e.g. C:\work\snack-web)
- 2 File → New File → save as snack.html
- 3 Paste the code on the right and save (Ctrl+S)
- 4 Double-click the file to open it in a browser

Opening it directly gives a CORS error.
Your file and the server (7901) are different origins, so the browser blocks it as a cross-origin request.

Fix — put it in the server folder and open it by the server address:

```
Save it as aidot-express/public/demo/snack.html  
→ http://localhost:7901/public/demo/snack.html
```

Then it is the same origin and nothing is blocked.

Three things to remember

1

load axios

One `<script>` line and you are ready.
Nothing to install.

2

axios.get(url)

is the errand "go to this address".
The answer arrives in `res.data`.

3

await and async

means "wait until the answer arrives".

How is it different from `fetch`?
`axios` unpacks the JSON for you.
`fetch` needs an extra `res.json()` call.

File

The full code to paste is on the next page

Step 7 (2) — the full code to paste

snack.html — copy the whole thing below

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="utf-8" />
  <title>Snack list</title>
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
</head>
<body style="font-family: sans-serif; padding: 24px">

  <h1>Snack list</h1>
  <button onclick="loadSnacks()">Load</button>
```

```
async function loadSnacks() {
  try {
    const res = await axios.get('/api/snack/');
    const list = res.data.data ?? res.data;
    document.getElementById('rows').innerHTML = list
      .map(s => '<tr><td>' + s.id + '</td><td>' + s.name +
        '</td><td>' + s.price + '</td></tr>')
      .join('');
  } catch (e) {
    alert('Could not load: ' + (e.response?.status ?? e.message));
  }
}
</script>
```

How to read it

1

The one script line at the top

lets you use axios without installing anything.

You need an internet connection.

2

res.data.data ?? res.data

If the server wraps it in { code, data }, take the inside;

A plain array is used as-is.

3

try / catch

A failure does not freeze the page.
tells you what went wrong.

Nothing appears?

- is the server running (npm start)?
- did step 5 create /api/snack?

File

Save to aidot-express/public/demo/snack.html → open localhost:7901/public/demo/snack.html

Step 8 (1) — create a Vue 3 project

Three commands and you are set — no TypeScript

1 Create the project

Command prompt

```
C:\work> npm create vue@latest snack-vue -- --router --pinia
```

Command prompt

```
VITE v8.1.5 ready in 512 ms  
  
→ Local: http://localhost:5173/  
→ press h + enter to show help
```

2 Skip the questions

-- --router --pinia

How to tell it worked

--typescript false

No questions

src/main.js

Do not use

Why two terminals?

7901 = the server (data)
5173 = the screen

Keep two terminals open side by side.

3 Install, then add axios

Command prompt

```
C:\work> cd snack-vue && npm install && npm install axios && npm run dev
```

If npm install fails?

Check that you are on Node 22 or newer.

If the port is taken, vite uses 5174.

Steps

Order: npm create vue@latest snack-vue -- --router --pinia → npm install → npm install axios → npm run dev

Step 8 (2) — only two files to edit

Two files to edit — vite.config.js and views/HomeView.vue

Folder layout (as generated)

snack-vue/	
package.json	axios was added here
src/	
main.js	Entry point — as is
vite.config.js	1 Add the proxy
node_modules/	npm install creates it
stores/	Leave it alone
components/	Leave it alone
views/HomeView.vue	2 Paste the code here
App.vue · router/	As generated

Edit two files. Leave the rest as generated.

File to edit (1) — vite.config.js

```
export default defineConfig({
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:7901',
        changeOrigin: true,
      },
    },
  },
  // leave plugins and resolve alone
})
```

Without this?

The screen (5173) and server (7901) differ, so the browser blocks it as cross-origin (a CORS error).

With a proxy the browser sees one origin.

Screen
localhost:5173



Proxy
(vite)



Server
7901

Step 8 (3) — HomeView.vue (full)

src/views/HomeView.vue — paste it as is (axios comes in via import)

```
<script setup>
import { ref, onMounted } from 'vue'
import axios from 'axios'

const snacks = ref([])
const error = ref('')
const loading = ref(false)

async function loadSnacks() {
  loading.value = true
  error.value = ''
  try {
    const res = await axios.get('/api/snack/')
    snacks.value = res.data.data ?? res.data
  } catch (e) {
    error.value = 'Could not load: ' + (e.response?.status ?? e.message)
  } finally {
    loading.value = false
  }
}

onMounted(loadSnacks)
</script>
```

```
<template>
  <main style="padding: 24px">
    <h1>Snack list</h1>
    <button :disabled="loading" @click="loadSnacks">
      {{ loading ? 'Loading...' : 'Reload' }}
    </button>
    <p v-if="error" style="color: crimson">{{ error }}</p>
    <table v-else border="1" cellpadding="6">
      <thead>
        <tr><th>No</th><th>Name</th><th>Price</th></tr>
      </thead>
      <tbody>
        <tr v-for="s in snacks" :key="s.id">
          <td>{{ s.id }}</td>
          <td>{{ s.name }}</td>
          <td>{{ s.price }}</td>
        </tr>
      </tbody>
    </table>
  </main>
</template>
```

How to read it

1

import axios from 'axios'

It comes from npm, not a CDN — imported inside script setup.

2

ref() and onMounted()

Put it in a ref and the screen redraws itself, onMounted loads once as soon as it opens.

3

try / catch / finally

A failure does not stop it, and always clears loading.

Nothing appears?

- did you add the proxy to vite.config.js?
- are both 7901 and 5173 running?

Confirm

Check: snacks in the table means it worked · if not, press F12 → Network and look at /api/snack

Step 8 — check the Vue screen (3)

Open `http://localhost:5173` and you get a screen with two menus at the top

How is this different from the HTML page?

- screen and data are separated
- the table refreshes itself
- several screens (addresses)

Something failed?
Press F12 and read the red text in the Console tab.
Server errors are in the Network tab.

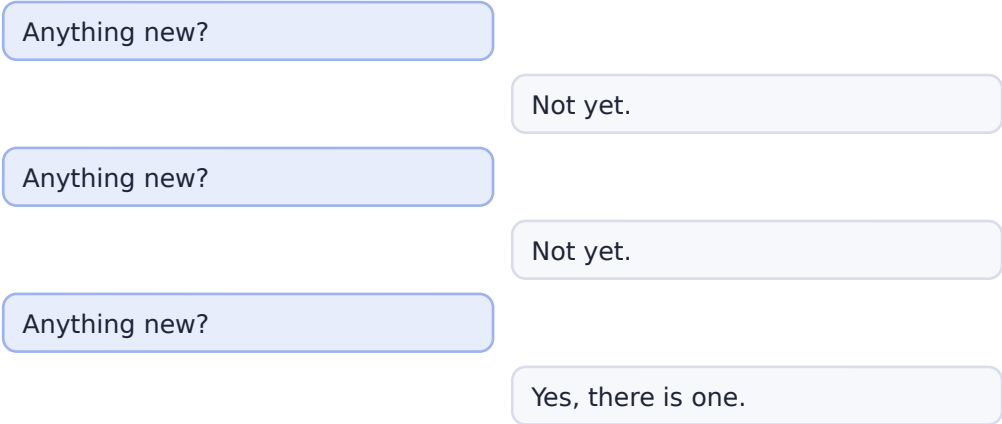
To ship the finished screen
`npm run build` → the dist folder

Steps Check: does the list appear · does the table grow after Add · is the response code 200/201

Step 9 — let the server tell you first (SSE)

So far the server answered when asked. Now it speaks first.

The old way — keep asking



Asking every 5 seconds is 17,280 times a day. Hard on the server, and news arrives up to 5 seconds late.

SSE — connect once and leave it



	Keep asking (polling)	SSE	In short
How fast news arrives	Late by the polling interval	Immediate	SSE is faster
Load on the server	Requests pile up	One connection	SSE is lighter
If it drops	Recovers on the next request	Reconnects and resends what was missed	The browser reconnects

Step 9 — three SSE endpoints already in the server

They already ship with aidot-express — nothing to build, just use them

GET

/api/events/stream?channel=guestbook

Connect and receive

POST

/api/events/publish

Publish news (to everyone)

GET

/api/events/stats

See how many are connected

That is all the server code there is

```
@SseMapping('/stream')                // ← this one line makes a stream
async stream(params, req) {
  req.sse.send({ event: 'hello', data: { message: 'connected' } });
  return params.channel || 'demo';    // return value = channel to subscribe to
}

// you can publish news from anywhere
sseHub.publish('guestbook', { message: 'a new entry arrived' });
```

What it handles for you



Reconnects when dropped

The browser reconnects itself after 3 seconds.



Resends what you missed

It sends only what happened while you were disconnected.



A keep-alive every 25 seconds

It keeps middleboxes from cutting the connection.



Split by channel

Different channel names never mix.

The console Load monitoring screen
is built on this. · design notes: docs/SSE_DESIGN.md

Step 9 — turn on live updates with one switch

No code needed — one switch on the controller edit screen

aidot-express admin consoleadmin

[Home](#)
[Dashboard](#)
[Controller](#)
[Service](#)
[SQL](#)
[API test](#)
[Logs](#)
[Users](#)



Live updates (SSE)
Switch it on and a subscribe address is added, publishing on every insert, update and delete.

Channel name
 Subscribe at `/api/snacks/events` [Try connecting](#)

Connected

14:31:02

change { "action": "created", "id": 7 }

Routes

Type	Method	Path	Handler	What it does
list	GET	/	list	List all
create	POST	/	create	Insert + publish
remove	DELETE	/:id	remove	Delete + publish
sse	GET	/events	events	← added for you

Click order

- 1 Turn on the Live updates switch
- 2 Check the channel name
(suggested for you)
- 3 Create controller
- 4 Try connecting

The moment you switch it on, an `@SseMapping` route and `sseHub.publish(...)` go in on their own.

If the controller requires auth, append after the address
`?access_token=TOKEN`
to the address.

Steps

Click order: Live updates → check the channel → Create controller → Try connecting → POST from another tab

Step 10 — make the screen live too (screen designer)

Build a screen on the API you made, and one switch makes it refresh itself

aidot-express admin consoleadmin

Home
Dashboard
Controller
Service
SQL
Screen designer
API test
Logs

Pick the data source

ControllerRoute

SnackControllerGET /api/snacks/

☒

Live refresh

This API has a live stream. Leave it on and every change makes the screen re-read the list on its own (no refresh needed).

Subscribe at: /api/snacks/events

The generated screen

Snack list

● LiveRefreshed

id	name	price	memo
3	Strawberry Milk	1300	← someone else just added this
2	Banana Milk	1200	After gym class

Click order

- 1 Screen designer → add a screen
- 2 Pick the data source for the list widget
- 3 Turn on live refresh
- 4 Export → npm run dev

For an API with no stream, this switch does not appear at all. (turn on Live updates on the controller first)

Open two windows and add a snack in one of them. the other list grows by itself.

Steps

Click order: pick the data source → turn on live refresh → export → check in two windows

Step 9 — see it live in two windows

Open the same page twice — send from one and it appears in the other at once

The diagram illustrates two browser windows connected via a WebSocket. Window A, titled "Window A — sending", is at `localhost:7901/public/demo/guestbook-lab.html`. It contains a "Channel name" field with "guestbook", a "Message to send" field with "Hello!", and a "Send a message" button highlighted with a red box. A green dot and the text "Connected" are shown next to the button. Below the input fields, a log shows "14:31:02 sent → Hello! (2 recipients)" and "14:30:55 connected to the server." Window B, titled "Window B — receiving (the Vue screen)", is at `localhost:5173/events`. It shows a green dot and the text "Connected". Below, a log shows "14:31:02 id 7 Hello!", "14:30:58 subscribed (channel=guestbook)", and "14:30:58 connected to the guestbook channel". A red arrow labeled "At once." points from Window A to Window B.

① Click Connect and the ● turns green

② What you send in one appears in the other immediately

③ Restart the server and it shows "reconnecting..." then attaches again by itself

Bonus

Leave a guestbook entry and the server tells you. From a terminal: `curl -N localhost:7901/api/events/stream?channel=guestbook`

Step 11 — split it into pages (pagination)

Twenty-four snacks will not fit on one screen. Show ten at a time.

```
-- the findAll you built so far
SELECT id, name, price, memo, created_at
FROM snack
ORDER BY id DESC;

-- returns all 24 rows at once.
-- With thousands of rows-- the screen crawls and the server strains.
-- Pagination
--   "just ten at a time"
-- is how you ask for that.
```

Two things you need

page

which page you want.

Counts from 1.

perPage

how many per page.

Usually 10 or 20.

With just these two,
the server slices it for you.

Next The SQL stays as it is; only how you run it changes — see the next page.

Step 11 — the server uses db.executeList

No new SQL. Change the execute function and LIMIT/OFFSET are added for you.

```
// service file
async listPaged(opts) {
  const sql = this.snackSql.get('findAll');
  return await db.executeList(sql, {}, {
    page:    opts.page,
    perPage: opts.perPage,
  });
}
// the shape it returns
// rows    : the 10 rows on this page
// header  : total 24, page 1,
//          perPage 10, totalPages 3
```

One line is the whole change

db.execute → db.executeList

Leave the SQL alone;

change only the execute call.
LIMIT and OFFSET are added for you.

rows and header

rows is this page,

header holds the numbers you need
to draw the pager.

SQLite and MariaDB both
work with the same function.

Tip

The SQL file is untouched — findAll is used as is.

Step 11 — one more address on the controller

Edit controller → Add route → pick listPaged and this code is written in

```
// controller file
@GetMapping('/paged')
async listPaged(params) {
  return this.snackService.listPaged({
    page:    Number(params.page)    || 1,
    perPage: Number(params.perPage) || 10,
  });
}
// new address
// GET /api/snack/paged?page=1&perPage=10
```

Why Number(...)?

Values from the URL are text —

turns "1" into the number 1.

|| 1
means page 1 when it is missing.

The old address stays

Keep GET /api/snack/ as well.

Some places still need everything.

Pick a type and
the controller and service both get code.

Steps Add route → type listPaged → Save controller

Step 11 — check that it really splits

In API test, change page and send again (assuming 24 snacks)

```
GET /api/snack/paged?page=1&perPage=10
→ 10 rows · totalPages 3

GET /api/snack/paged?page=2&perPage=10
→ 10 rows

GET /api/snack/paged?page=3&perPage=10 → 4 rows    (24 - 20 = 4)
-- the last page returns only what is left.
```

Three things to check

Do the numbers add up?

- 1 row count = perPage
- 2 header.total = total rows
- 3 the last page has the remainder

If they do not,

Check the ORDER BY in your SQL.

Without a sort,
the order shuffles.

Once this works,
you're ready to wire it to a screen.



24 rows at 10 each is 3 pages. Four rows on the last page is right.

Step 11 — add a pager in the screen designer

Screen designer → drop in the List (Paged) widget and point it at the address you just made

Click order

- 1 Screen designer → open the project
- 2 Drag List (Paged) from the widgets on the left
- 3 Data source on the right → Edit
- 4 `SnackController` → `route listPaged`
- 5 Check `perPage` (10 by default) 6 Preview and check the page buttons

How is it different from List?

List

draws every row it receives.

there is no pager.

List (Paged)

First, previous, numbers, next, last

The pager is inside the widget.
You do not build them.

The address must be `listPaged`.
A plain list address returns no header, so

Tip

The pager needs `totalPages` to draw the page buttons.

Step 12 — file upload (the approach first)

By default a file arrives as text inside JSON — multipart works too (later)

Browser

```
FileReader.readAsDataURL(file)
↓
"data:image/png;base64,iVBORw0..."
↓
{ fileName: "snack.png",
  fileBase64: "iVBORw0..." }    ↓ an ordinary POST · JSON server
Buffer.from(fileBase64, "base64")
↓
saved to public/uploads/
```

Why do it this way?

The controller looks like any other

One params argument is all you take.

With multipart, only that controller is configured differently.

You are already using it

the MCI definition spreadsheet

and the backup zip
all work this way.

The body limit is 10MB.
base64 grows the payload by a third, so

Note For bigger files, raise `BODY_LIMIT` in `.env`.

Step 12 — start with an upload that exists

Before building one, try an upload the console already has — the approach becomes obvious

```
1 Create an MCI controller
  MCI channel → Create MCI controller
  Upload the interface spreadsheet and
  a whole controller is generated.

2 Restore a backup
  Backup/restore → upload the zip and settings and code come back.
-- both work the same way.
```

See it for yourself

F12 → Network

Open the request body and

you can see fileBase64.
The file went as text.

A file to practise with

Upload

docs/mzd_deptmmt_l01.xls
into the MCI wizard.

Try this first and
the next page reads much easier.

Tip

Clicking through something that already works beats reading about it.

Step 12 — build the upload controller yourself

Create an empty controller with New controller and paste this in

```
@PostMapping('/')
async upload(params) {
  const { fileName, fileBase64 } = params;

  // 1 never trust the name as given
  const safe = path.basename(fileName)
    .replace(/[^\w.\- ]/g, '_'); // 2 text back into a file
  const raw = fileBase64.includes(',')
    ? fileBase64.split(',').pop() : fileBase64;
  const buf = Buffer.from(raw, 'base64');
  fs.writeFileSync(dir + '/' + safe, buf); return { data: { fileName: safe } };
```

Two things you must do

1 Sanitise the file name

A name like "../etc/passwd" writes outside the project.
Strip the path with `basename`.

2 Never overwrite

A repeated name gets -2, -3.
An overwrite cannot be undone.

For a screen, use the HTML lab page — see <public/demo/upload-lab.html>.

Done An uploaded file opens at `/uploads/<filename>`.

Step 12 — two upload styles · multipart

The same file can travel two ways. You only need to know which to use when.

```
# 1 base64 (JSON) — what you have used so far
POST /api/uploads/
Content-Type: application/json
{ "fileName": "snack.png",
  "fileBase64": "iVBORw0..." }

# 2 multipart — what a browser <form> uses
POST /api/uploads/multipart
Content-Type: multipart/form-data

curl -F "file=@snack.png" \
      -H "Authorization: Bearer <TOKEN>" \
      .../api/uploads/multipart

# both return the same shape
# { fileName, size, url }
```

Which one?

Use base64 when

a screen already exchanging JSON
sends one file alongside.
The body grows by a third
(a 3MB file goes out as 3.8MB)

Use multipart when

a browser <form>, curl -F, or
another system is sending.
It goes out at its real size.
The limit is UPLOAD_MAX_BYTES in .env

Both styles sanitise the name and
Duplicates get -2, -3.

Recap

base64 when it travels with JSON, multipart when it is just a file. The saving rules are the same.

Logs (1) — read what the server wrote

This screen opens the files the server wrote: Logs → kind → month → file

aidot-express admin console

admin

Home

Dashboard

Controller

Service

SQL

API test

Logs

Users

GeneralSQL

general = what the server did · sql = SQL only, when LOG_SQL=true

1 Folder (by month)

2026-08

2026-07

2 File (one per day)

File	Size	Modified
2026-08-24. log	71 KB	Today 14:31
2026-08-23. log	224 KB	Yesterday 23:59
2026-08-22. log	351 KB	2 days ago

3 Contents

```
2026-08-24 14:30:11 [INFO] [Route] GET /api/snacks/ → SnackController.list
2026-08-24 14:30:11 [INFO] SnackService::list called
2026-08-24 14:30:55 [INFO] [sse] subscribed id=sse_mt6c... channel=guestbook (2 total)
2026-08-24 14:31:02 [WARN] [404] GET /api/snack - Not Found
```

The order never changes

- 1 Kind (general / sql)
- 2 Month folder (2026-08)
- 3 File (one per day)
- 4 Read it

[INFO] is just news,
[WARN] is a caution,
[ERROR] is a real problem.

Start with the red ERRORS.

The files are in the log folder too.
On an installed build, use
Open log folder takes you straight
there.

Steps

Click order: Logs → kind tab (general) → folder (2026-08) → file (today) → read

Logs (2) — just the lines you want

Tens of thousands of lines pile up daily. The quick-query buttons in Log explorer pick them out.

The screenshot shows the 'aidot-express admin console' interface. On the left is a sidebar with navigation links: Home, Dashboard, Controller, Service, SQL, API test, **Logs** (highlighted in red), and Users. The main area has a search bar containing 'SnackService' (highlighted with a red box), a 'Search' button, and a 'Search results zip' button. Below the search bar are filters: 'Regex' (disabled), 'Matching lines only' (enabled, highlighted with a red box), and 'Level' (set to 'ERROR', highlighted with a red box). The results show '12 matches · showing 12 lines after filtering'. The log entries are as follows:

```
2026-08-24 14:30:11 [INFO]    SnackService::list called
2026-08-24 14:30:24 [INFO]    SnackService::create called
2026-08-24 14:30:24 [ERROR]   SnackService::create failed
ER_NO_SUCH_TABLE: Table 'aidot_express.snack' doesn't exist
2026-08-24 14:31:40 [INFO]    SnackService::list called
2026-08-24 14:32:02 [INFO]    SnackService::remove called
```

Search for things like

- a controller name
- a service name
- an address (/api/snacks)
- an error code (starts with ER_)

Turn on "matching lines only" and only matches remain, which is far easier to read.

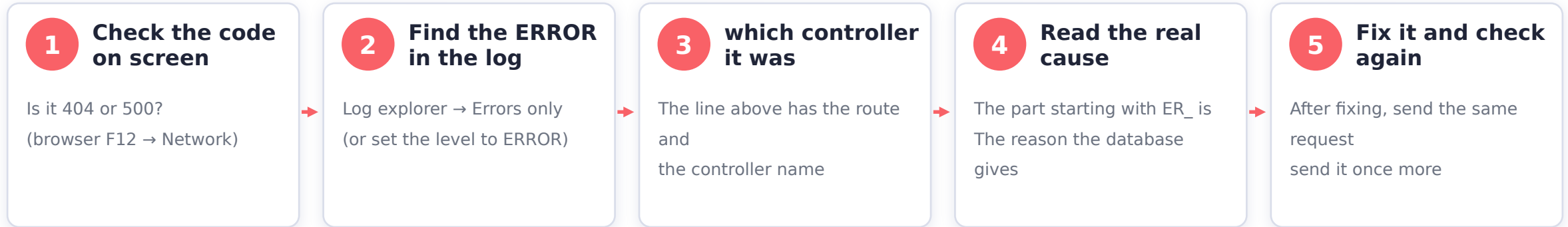
Click "Search results zip" and you get the results as a file to send to someone else.

Steps

Click order: type a term → matching lines only → level ERROR → Search → zip if you need it

Logs (3) — what to do when something errors

Do not stop at "it does not work" — these five steps lead to the cause



Read a real log

```
2026-08-24 14:30:24 [INFO] [Route] POST /api/snacks/ → SnackController.create
2026-08-24 14:30:24 [INFO] SnackService::create called
2026-08-24 14:30:24 [ERROR] SnackService::create failed
  ER_NO_SUCH_TABLE: Table 'aidot_express.snack' doesn't exist
  at SnackService.create (workspace/service/SnackService.js:38)
```

- Where?** SnackController.create → adding a snack
- What?** Table 'snack' doesn't exist → the table is missing
- How?** Go back to step 0, create the snack table, and it is fixed

Remember: logs run top to bottom in time. Once you find the ERROR, the few lines above explain why.

Part 3 : Who asked for what, and how it went

Step 14 — tracing (1): who, what, how

Management → Request tracing walks back through every request. No log grepping.

기록 16,093건 · 보관 · 30일 · 최대 200,000건

원가 실패했어요
오류 느린 요청부터 봅니다

요청 번호가 있어요
화면에 뜬 번호로 바로 찾습니다

누가 뭘 했는지 볼래요
사람을 고르고 행적을 봅니다

100 요청 10 실패 287ms p95 소요 18 서로 다른 경로 4분 기간

요청 추적

하나의 요청이 어떤 단계를 거쳤는지, 누가 무엇을 했는지 되짚습니다

요청 ID 로바로 찾기

a3f9c210 — 오류 화면에 표시된 번호

찾기 자주 찾는 것

응답 헤더 X-Request-Id 와 로그 줄의 [번호] 가 같은 값입니다.

⊗ 서버 오류(5xx) ⚠ 실패한 요청(4xx+) ⌚ 1초 넘는 요청 🗑 삭제 행위

사용자 내 요청만

메서드 전체

경로 포함 /api/admin/users

상태 ≥ 400

소요 ≥ (ms) 500

🔍

요청 목록 100

시각	요청	상태	소요	이유
10:31:26	GET /api/admin/trace ↳ admin · 1d256c5c	200	30ms	
10:31:26	GET /api/admin/trace/status ↳ admin · 854faa01	200	13ms	
10:31:25	admin · 6581da6c	200	5ms	

Pick one of the three cards

Something failed → start with errors and slow requests

Got a request id → go straight to it

Who did what → per-person trail

It opens on "my requests"

The numbers on top: requests, failures, p95 duration, distinct paths.

Turn off "my requests only" to see everyone.

Four common searches

[Server errors (5xx)] [Failed requests (4xx+)]

[Over 1 second] [Deletions]

Clicking one fills the filters and changes the list

Steps

Request tracing → pick a card → Failed requests (4xx+) → click a row

Step 14 — tracing (2): click a row for the story

Click POST /api/books and its story and steps open on the right

The screenshot shows a request tracing interface. On the left, there is a table with columns: 시간 (Time), 요청 (Request), 상태 (Status), 소요 (Duration), and 이유 (Reason). The table lists three requests: a DELETE request to /api/books/:id with status 200 and duration 4ms, a GET request to /api/books/:id with status 200 and duration 5ms, and a PUT request to /api/books/:id with status 200 and duration 4ms. On the right, there is a detailed view of a request. The top bar shows the request ID d0b0adb6, a button to view the log, the status 201, and buttons for story, waterfall, and raw views. The main content area shows the story of the request: 'student1 님이 책을 만들었습니다 — 성공 (6ms, SQL 1회, 1건)' followed by the timestamp and trace ID. Below the story, there is a waterfall chart showing the steps of the request: +0ms request arrives, +1ms BookController.create receives the request, +3ms SQL book:insert executes (1ms, 1 row), +4ms BookService.create returns the response, and +6ms BookController.create finishes the request (5ms).

시간	요청	상태	소요	이유
10:31:19	DELETE /api/books/:id student1-7953f1e7	200	4ms	
10:31:19	GET /api/books/:id student1-ecfc1915	200	5ms	
10:31:19	PUT /api/books/:id student1-054bc965	200	4ms	

student1 님이 책을 만들었습니다 — 성공 (6ms, SQL 1회, 1건)
2026. 8. 27. 오후 10:31:19 · 127.0.0.1 · trace 61a1a5ed3...

+0ms POST /api/books 요청이 들어왔습니다
+1ms BookController.create가 요청을 받았습니다
+3ms SQL book:insert 실행 (1ms) → 1건
+4ms BookService.create를 호출했습니다
+6ms BookController.create가 처리를 마쳤습니다 (5ms)

Story = one sentence: "student1 created a book — success (6ms, 1 SQL, 1 row)"

Who, what and how it went, in one line. If it failed, the reason is right there.

Steps = the path the request took

+0ms request arrives → +1ms BookController.create → +3ms SQL book:insert (1ms, 1 row) → +3ms BookService done → +4ms response 201

It shows where the time went and which SQL touched how many rows.

Three views

Story in plain words · Waterfall as time bars · Raw as JSON

The green number at the top right is the HTTP status (201).

Try it

- 1 Create a snack (or book) from API test
- 2 Click the POST row in Request tracing
- 3 Read the story and the steps

Why it helps

You used to grep whole log files. Now you find that request and see where it stopped.

Try it

Click POST /api/books → read the story → Waterfall for time bars → Raw for the JSON

Step 14 — tracing (3): follow one person

The "who did what" card → pick a person and their trail opens in time order

 **student1 님의 행적** [닫기](#)

 **2026. 8. 27. 오후 10:28:00 — 오후 10:31:19** 24건

- 오후 10:28:00 • student1 님이 책을 조회했습니다 — 성공 (9ms)
- 오후 10:28:00 • student1 님이 책을 조회했습니다 — 성공 (9ms)
- 오후 10:28:00 • student1 님이 책을 만들었습니다 — 성공 (6ms)
- 오후 10:28:00 • student1 님이 책 한 건을 수정했습니다 — 성공 (4ms)
- 오후 10:28:00 • student1 님이 책 한 건을 조회했습니다 — 성공 (6ms)
- 오후 10:28:00 • student1 님이 책 한 건을 삭제했습니다 — 성공 (4ms)
- 오후 10:29:14 • student1 님이 책을 조회했습니다 — 성공 (11ms)
- 오후 10:29:14 • student1 님이 책을 조회했습니다 — 성공 (6ms)

How do you open it?

- 1 The "who did what" card
- 2 Click a name (student1) under "find by person"
- 3 "student1's trail" opens on the right

Shown in groups

It groups one sitting: "10:28:00 pm — 10:31:11 · 18 requests". A gap over 30 minutes starts a new group.

Click a row in the trail and that request opens.

Set up — a second account

Create student1 (role user) under Users, then create, edit and delete a book as them. Then pick student1 here.

Steps Who did what → find by person: student1 → read the groups → click a row → details

Step 14 — tracing (4): find by id

One eight-character id on the error screen, the response header and the log finds the same request anywhere

요청 추적 하나의 요청이 어떤 단계를 거쳤는지, 누가 무엇을 했는지 되짚습니다

요청 ID 로바로 찾기

d0b0adb6 찾기 자주 찾는 것

응답 헤더 X-Request-Id 와 로그 줄의 [번호] 가 같은 값입니다. 🚫 서버 오류(5xx) ⚠️ 실패한 요청(4xx+) 🕒 1초 넘는 요청 📊 식제 행위

사용자: ☐ 내 요청만 ☐ 전체 메시지: 전체 경로 포함: /api/books 상태 >= 400 소요 >= (ms) 500

student1 🔍

요청 목록 24 📄 d0b0adb6 📄 이 요청의 로그 보기 201 🗨 이야기 🔍 폭포수 📄 원자료

거르기 14,277줄 · 1파일

유형

app	11,455
http	1,996
sql	577
migration	134
boot	76
auth	39

수준

INFO	10,876
DEBUG	3,228
WARN	149
ERROR	24

요청 많은 순

4줄

```
npm run logs d0b0adb6
```

```
22:31:19.774 [INFO] app d0b0adb6 [BookController.js:33:14] BookController::create 호출됨 -> params={"title"...}
22:31:19.775 [INFO] app d0b0adb6 [BookService.js:30:14] BookService::create 호출됨
22:31:19.776 [DEBUG] sql d0b0adb6 [db.js:338:36] [DB:mariadb] INSERT INTO sample.book (title, author... 1.2ms
22:31:19.780 [DEBUG] http d0b0adb6 [server.js:129:7] 127.0.0.1 user=- "POST /api/books HTTP/1.1" 201 136 "...
```

1 "I have a request id" → paste it → Find

2 "Show this request in the log" narrows Log explorer to that id

Where do you find the id?

- the X-Request-Id response header
 - the id on an error screen
 - the id in front of a log line
 - the grey id in the tracing list
- They are all the same id.

The two screens connect

Request tracing → "Show this request in the log" → Log explorer, that request only

The ↗ icon beside a log line goes back to the request details

Anything the steps do not show (a log.info inside a service) is in the log.

Try it

- 1 Send anything from API test and copy the X-Request-Id header
- 2 Paste it into "I have a request id" and click Find
- 3 "Show this request in the log" takes you there

Steps

Copy the id → "I have a request id" → Find → "Show this request in the log" → ↗ to come back

Step 14 — tracing (5): when nothing shows up

A missing request is usually one of these four reasons

Symptom	Why	What to do
My GET request is not in the list	A fast anonymous GET is kept in memory for a moment, not written to the database	You can still find it by id (last 1,000). Errors, slow requests and signed-in requests are stored.
I cannot see anyone else	"My requests only" is switched on	Turn the switch off and type a name in the user box
A yellow banner appears at the top	The server is busy and is recording less for a while	Look again shortly — errors and slow requests are still recorded meanwhile
"Show this request in the log" is empty	Admin API lines are hidden from the log, and a fast request with no SQL has nothing to write.	The steps are in the request details. For the log too, set LOG_ADMIN=true in .env
Still there after a restart?	Stored requests are kept 30 days in request_traces	Use "Run cleanup now" in the top banner to clear old ones

In one line: who/what/how is Request tracing, what the server said is Log explorer. One id moves between the two.



Tracing: story and steps · trail per person · find by id · jump to the log

When it does not work — start here

Almost every problem is one of these eight

404 Not Found

No such address

Check the trailing / after /api/snacks, and the spelling

500 + Table doesn't exist

The table is missing

Check that you ran the CREATE TABLE from step 0

500 + Unknown column

The column names differ

Make the names in the SQL match the table columns

An empty list []

There is no data

That is fine — POST one snack and try again

401 Unauthorized

Sign-in required

Check whether "require auth" is ticked on the controller

The page will not open

The server is not running

Check that npm start is running in a terminal

CORS error (red text)

Blocked because the origins differ

For Vue check the proxy in vite.config.js; for a deployed build set CORS_ORIGIN

No SSE updates arrive

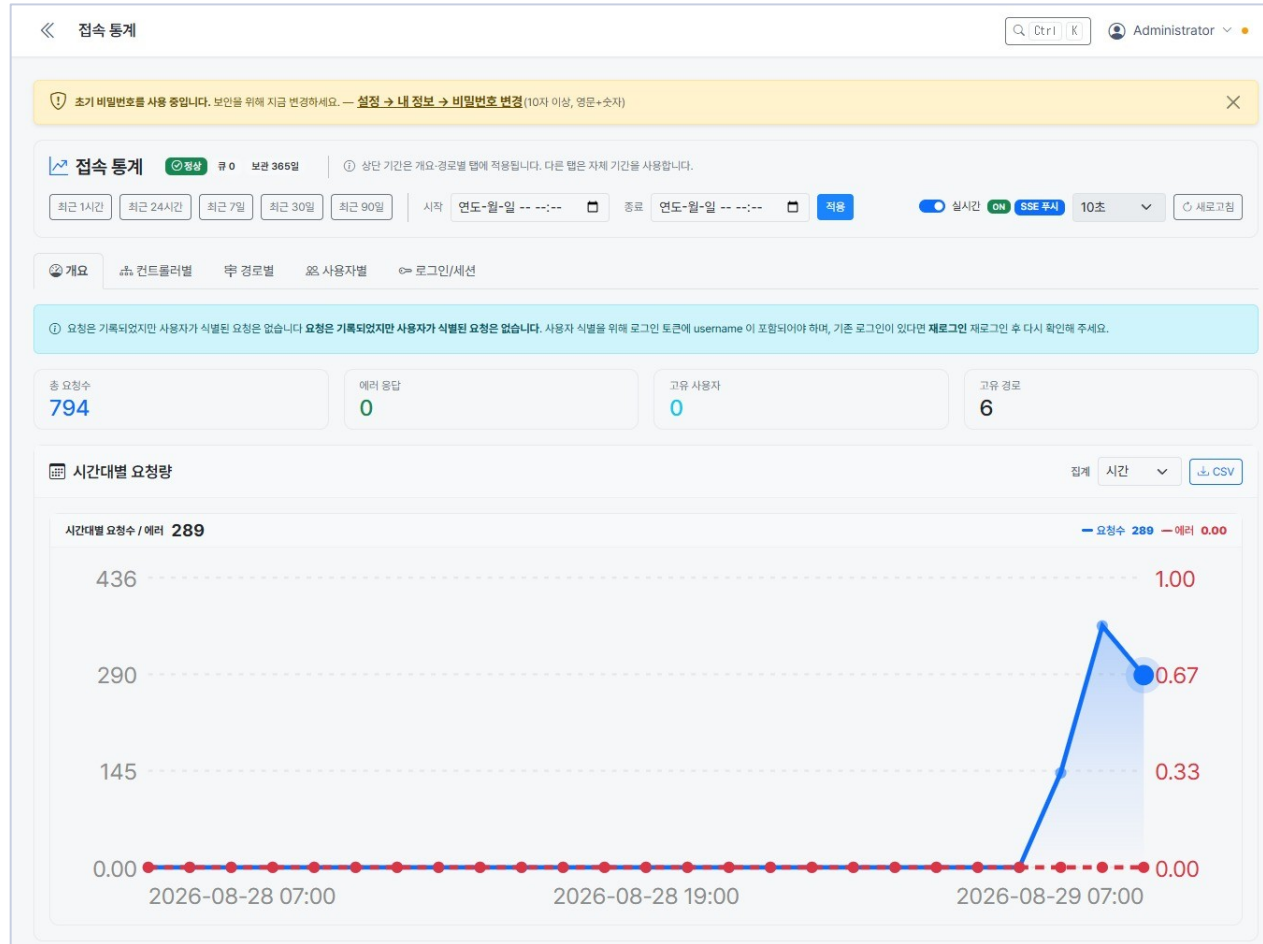
Not connected

Check the ● is green and that you appear in /api/events/stats

Still stuck? Look for red lines under Logs in the left menu — they say what went wrong.

Step 15 (1) — see who used how much

Access stats, near the bottom of the left menu, gathers every request so far



The period buttons on top

Last hour · Last 24 hours · Last 7 days ...

For your own dates, pick a start and end and click Apply.

Turn on Live and

the numbers refresh themselves.

The 10s beside it is the refresh interval.

Five tabs

Overview · by controller · by path ·

by user · sign-ins and sessions

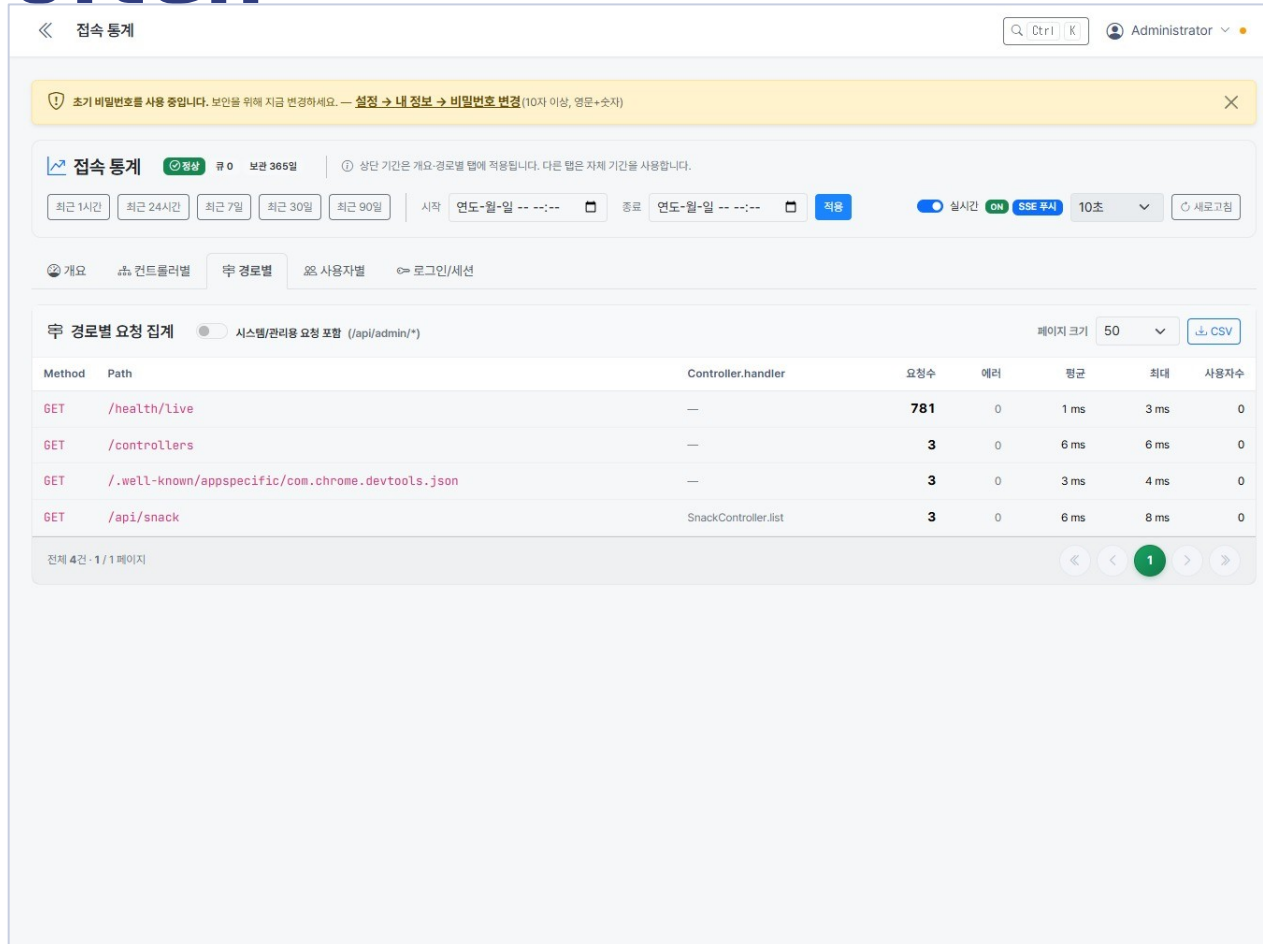
The overview shows request counts and a graph over time.

Click

Access stats in the left menu → pick a period → step through the tabs

Step 15 (2) — which feature was called how

By controller groups by feature; by path groups by address
often



The screenshot shows a web application monitoring dashboard. At the top, there's a navigation bar with a search bar and a user profile. Below it, a yellow banner contains a security warning. The main content area has a sidebar with navigation links and a main panel displaying a table of requests. The table has columns for Method, Path, Controller.handler, 요청수 (Request Count), 에러 (Error), 평균 (Average), 최대 (Maximum), and 사용자수 (User Count). The table is filtered by '경로별 요청 집계' (Request Aggregation by Path) and shows 4 items. The first item is a GET request to /health/live with 781 requests, 0 errors, 1 ms average, 3 ms maximum, and 0 users. The second item is a GET request to /controllers with 3 requests, 0 errors, 6 ms average, 6 ms maximum, and 0 users. The third item is a GET request to /.well-known/appspecific/com.chrome.devtools.json with 3 requests, 0 errors, 3 ms average, 4 ms maximum, and 0 users. The fourth item is a GET request to /api/snack with 3 requests, 0 errors, 6 ms average, 8 ms maximum, and 0 users. The table is paginated with 4 items per page.

Method	Path	Controller.handler	요청수	에러	평균	최대	사용자수
GET	/health/live	—	781	0	1 ms	3 ms	0
GET	/controllers	—	3	0	6 ms	6 ms	0
GET	/.well-known/appspecific/com.chrome.devtools.json	—	3	0	3 ms	4 ms	0
GET	/api/snack	SnackController.list	3	0	6 ms	8 ms	0

By controller

like SnackController 787,
which feature was used how much,
on one line.

System requests are excluded

Health checks and console screens
are not counted. To include them,
turn on "include system requests".

as CSV or Excel.

exactly as shown,
You can download the table you see

How to read Requests · 5xx · 4xx · average and max response time · users — a slow feature has a high average

Step 15 (3) — is the "by user" tab empty?

To record who called, the controller needs auth turned on

Why is it empty?

The /api/snack you built so far is open to anyone.

With no way to know who called, there is nothing to group by user.

Then names are recorded

1 Open the controller

Controllers → the pencil icon on SnackController

2 Turn on "require auth"

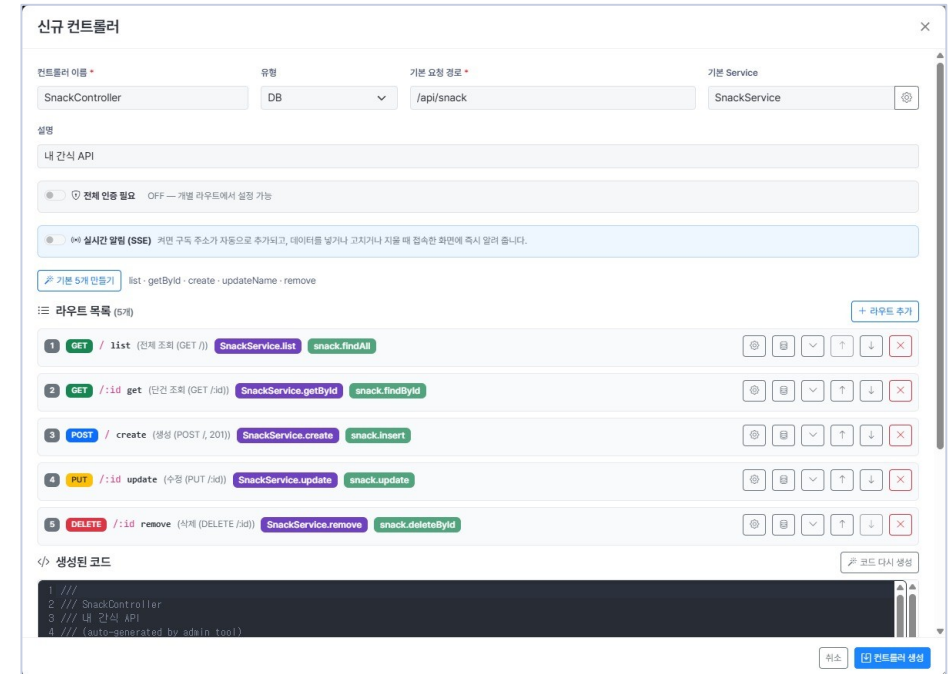
Turn it on and every route requires a signed-in caller

3 Create/save the controller

@Auth() is added to the code

4 Send a token when you call it

API test attaches it for you when "auto-attach token" is on



```
@Controller('/api/snack')
@Auth() // ← this line appears
export default class SnackController { ... }
```

The point Auth is what puts names in the by-user tab — without a token they pile up as "anonymous"

Step 15 (4) — export per-user stats

See what one person used and how much, then download it as Excel

접속 통계

초기 비밀번호를 사용 중입니다. 보안을 위해 지금 변경하세요. — [설정](#) → [내 정보](#) → [비밀번호 변경](#) (10자 이상, 영문+숫자)

접속 통계

기간: 최근 1시간, 최근 24시간, 최근 7일, 최근 30일, 최근 90일

시작: 연도-월-일 -- --:-- 종료: 연도-월-일 -- --:-- 적용

실시간 ON SSE 무시 10초 새로고침

개요 컨트롤러별 루트 경로별 사용자별 로그인/세션

컨트롤러별

사용자 아이디 (선택) 콘솔 제외 CSV 엑셀

컨트롤러	호출	라우트	사용자	4xx	5xx	평균(ms)	최대(ms)	마지막 호출
(기타)	787	3	0	3	0	1	6	2026-08-29 07:48:12
SnackController	3	1	0	0	0	6	8	2026-08-29 07:47:12

The by-user tab

Per person: requests, errors, average response, first and last seen — all on one line.

For one person only,

The buttons above the by-controller tab type the name into the user id box. shows only what that person used.

Click Excel and

keep the period and filters you are viewing you get an .xlsx file.



Pick a period → (type a user id if you need one) → Excel or CSV

Step 15 (5) — follow what one person did

Stats answer "how much", tracing answers "what and when" — read all three together

1 When they signed in — Access stats → Sign-ins

Sign-in and sign-out times, and from which IP.
If something looks wrong you can end the session right there.

2 What they called — Access stats → by user → click the name

their paths appear, most-called first.
(Raw log on the same screen is line by line, in time order)

3 What happened then — Request tracing

Pick one request and follow controller → service → SQL —
you see how many ms each step took.

Three screens, one record

Access stats = counted and grouped
Raw log = line by line
Request tracing = what happened inside one request
Access stats are kept 365 days,
request traces 30 days by default.

For an audit trail

Pick a period, export to Excel and
it comes out as a single sheet.
Type a user id for just that person.

An API without auth records no name

Steps

Sign-ins (when) → by user (what) → Request tracing (what happened inside)

Step 16 (1) — calling an API that requires auth

Attach the token you got when signing in. That is also what records who called.

What is a token?

It is like a pass. Sign in and the server hands you a long string.
Send that string with each request and the server knows who you are.
It lasts 15 minutes; after that, sign in again or refresh it.

```
# 1 sign in and get a token
POST /api/admin/auth/login
{ "username": "admin", "password": "admin1234" }

→ data.accessToken holds a long string
```

```
# 2 attach it — one header line
GET /api/snack
Authorization: Bearer eyJhbGciOi...

# without it you get 401 (unauthorised)
```

Do not leave out Bearer

The Authorization value is
Bearer, a space, then the token.
The token alone will not get you through.

Why bother turning it on

- it stops anyone from calling it
- names appear under Access stats → by user
- when something breaks you can see who did what

A token is as good as a password

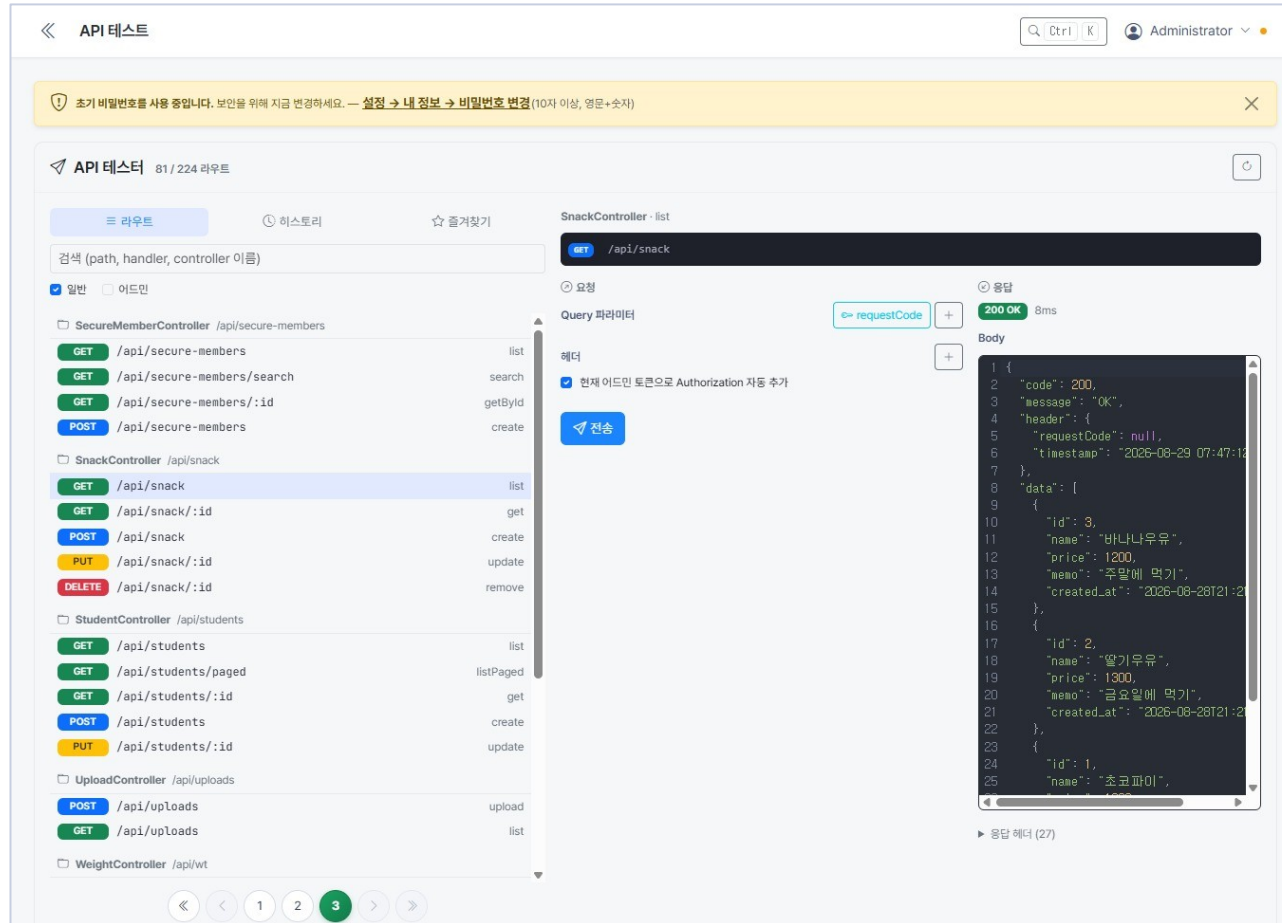
Never paste it into chat or an issue.
It expires in 15 minutes,
Until then it lets anyone act as that person.

**The
point**

Sign in → get accessToken → send Authorization: Bearer <token>

Step 16 (2) — calling it from API test

No copying tokens — leave it on and it attaches one for you



1 API test in the left menu

The controllers you built are listed

2 Pick the route to call

[GET] /api/snack → list

3 Turn on auto-attach token

It attaches the signed-in user's token

4 Send

The response appears below

If you get 401,

- auto-attach token is off, or
 - your session expired (sign in again)
- 403 means the role (admin/user) is not enough.

Click

API test → pick a route → auto-attach token ON → Send

Step 16 (3) — calling it from Postman

Outside the console you fetch the token yourself and put it in a header

1) Sign in first and get a token

```
POST http://localhost:7901/api/admin/auth/login
Body → raw → JSON
{ "username": "admin", "password": "admin1234" }
```

Copy `data.accessToken` from the response

2) Put it in the header when you call the API

```
GET http://localhost:7901/api/snack
Headers
Key    : Authorization
Value  : Bearer {{token}}
```

Setting Type to Bearer Token on the Authorization tab does the same.

If copying it each time is tedious,

```
// in the sign-in request, Scripts → Post-response
const d = pm.response.json().data;
pm.environment.set("token", d.accessToken);

// use {{token}} in your other requests
Authorization: Bearer {{token}}
```

set up an Environment and

One sign-in and every other request goes through.

The token expires in 15 minutes

On a 401, send the sign-in request again.



Sign in → copy accessToken (or save it by script) → call with Authorization: Bearer <token>

Step 17 (1) — where does the token come from?

Signing in gets you a long string from the server. You take it and use it.

1 Send a sign-in request

```
POST /api/admin/auth/login
Content-Type: application/json

{ "username": "admin", "password": "admin1234" }
```

2 The token is inside the response

```
{
  "code": 200,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ", ← this one
    "refreshToken": "...",
    "user": { "username": "admin", "role": "admin" }
  }
}
```

How to get it, per tool

```
// browser · Node
const r = await fetch('/api/admin/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username, password }) });
const token = (await r.json()).data.accessToken;
```

```
# Postman — the sign-in request, Scripts → Post-
response
const d = pm.response.json().data;
pm.environment.set("token", d.accessToken);
```

A token is as good as a password

Never paste it into chat or an issue. It expires after 15 minutes.

Summary

Sign in → keep `data.accessToken` from the response

Step 17 (2) — where do you put the token?

One line in the request header. Only the place differs by tool.

Authorization: **Bearer** eyJhbGciOiJIUzI1NiIs...

There must be one space between Bearer and the token. The token alone will not pass.

Console API test

Turn on auto-attach token and
The signed-in user's token —
it is attached for you.

That is the easiest way.

Postman

Authorization tab →
set Type to Bearer Token
paste it in the token box

or use the `{{token}}` variable

In code

```
fetch(url, { headers: {  
  Authorization:  
    `Bearer ${token}` } })
```

axios takes the same line in headers

401 means a missing header or an expired token · 403 means the token is fine but the role is not

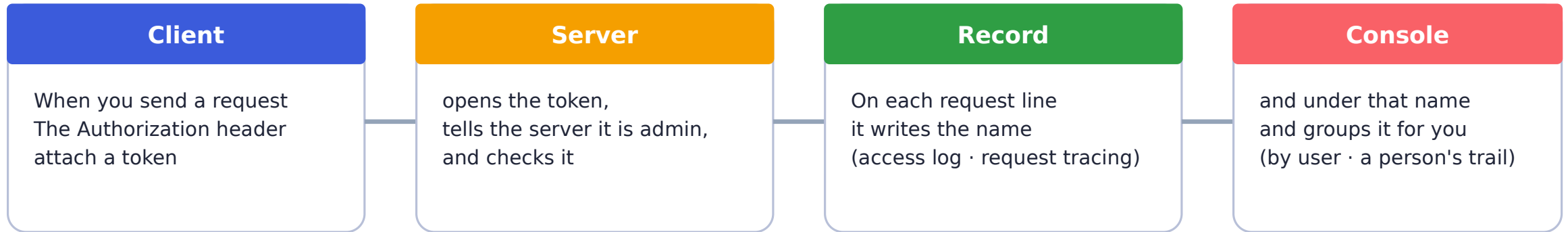
**Where it
goes**

Authorization: Bearer <token> in every request header

Step 17 (3) — so how does the console know

The name is inside the token, and the server records it on every request

who did it?



```
# the line the server writes (access log)
```

```
2026-08-30 07:12:03 admin GET /api/snack 200 6ms 10.0.0.31
```

```
↑ the name read from the token — without it there is no "who" to count
```

And without a token?

The name field is empty and it piles up as "anonymous" — there is no way to tell who.

The sign-in itself is recorded too

A sign-in happens before there is a token, so the id is recorded only on success.

**One
line**

Attach a token → the server knows the name → it is recorded → the console groups it by person

Step 17 (4) — follow a person in the console

This is how the two screens work together (drawn as they really look)

Aidot

Dashboard

Controller

Access stats

Request tracing

Logs

Access stats

Overview

By controller

By path

By user

User	Requests	Errors	Avg	Last seen
admin	214	1	6ms	07:12
nurse01	52	0	8ms	07:05

↑ click a person and their paths appear below

GET /api/snack128

POST /api/snack12

[] include system requests ← sign-ins etc.

Access stats = how much was used

Aidot

Dashboard

Controller

Access stats

Request tracing

Logs

Request tracing — admin's trail

Controller requests

System access

All

07:12:03GET /api/snack2006ms

07:12:01POST /api/snack20114ms

07:11:58GET /api/snack/32005ms

07:11:40GET /api/snack2007ms

↑ scroll up to load older records

opens the step-by-step details of that one request

Request tracing = what and when

order to use

Find the person in Access stats → follow their trail in Request tracing

What you did today

- ✓ Installed it (folder or installer) and created the snack table
- ✓ Opened five addresses from the console: SQL → service → controller
- ✓ Checked it in a browser, API test and Postman
- ✓ Built the same thing by hand with three files
- ✓ Wired up HTML and Vue 3 screens, and live SSE
- ✓ Added pagination (executeList) and file upload

What to try next

Add search

Add WHERE :kw to the list SQL and connect a Query Form widget

Lock it down

Tick "require auth" on the controller → only signed-in callers get through

Manage uploaded files

Add an upload list screen, delete, and image previews

Test as a set · back up

Check everything at once with Scenario test · download a zip from Backup/restore

Q&A -- mike.jung.global@gmail.com